

ARCHON AGENTIC OS 2026 — 전체 기능 목록

시스템 및 모듈 카탈로그, 아키텍처 개요와 기능 매트릭스

기준일: 2026 년 11 월 11 일 | 버전: v0.5.1(“Archon Hardened”) | 언어: 한국어

시스템 및 모듈 카탈로그, 아키텍처 개요와 기능 매트릭스

© Michael Keller

목차

1. Versionsstand & Release-Übersicht (v0.5.1) 3
2. System-Ansichten & Spezial-Studios (UI Views) 3
3. Token-Spar- & FinOps-Ökosystem (Kostenminimierung) 8
4. Kernel, Scheduling & Daemon-Architektur (Pillar 1) 12
5. Multi-Agenten-Prozessmodell, Hierarchie & Synthese (Pillar 2) 13
6. MCP-Treiber, Externe Dienste & Multimodale Werkzeuge (Pillar 3) 15
7. Gedächtnis, Datenbank-Hygiene & Obsidian 2nd Brain (Pillar 4) 17
8. Sicherheit, Zero-Trust Governance & System-Diagnose (Pillar 6) 19
9. Selbst-Evolution, Marktplätze & Auto-Dokumentation (Pillar 7) 20
10. Referenz-Matrix: Steuerbare Evey/Hermes-Plugins 21
11. Enterprise- & Resilienz-Architektur 23
12. Autonomie-, Sandbox- & Governance-Architektur 25
13. Performance-Core, Durchsatz-Skalierung & Token-Effizienz 27
14. Moderne Gedächtnis- & RAG-Architektur 28
15. Zuverlässige Ausführung & Determinismus (Durable Execution) 30
16. Sicherheit, Datenschutz & Enterprise-Guardrails 32
17. Selbst-Optimierung, Evaluation & Schwarm-Intelligenz 33
18. Säule 3: Wirtschaft & ERP (alpha) & Mandanten-Suite 35
19. Architektur-, Security- & Deployment-Features (v0.5.0 – v0.5.1) 35
20. Alphabetisches Glossar & Stichwortverzeichnis 36

© Michael Keller

1. 버전 현황 및 릴리스 개요(v0.5.1)

버전	코드네임	주요 아키텍처 및 기능 마일스톤
v0.5.1	Archon Hardened	모듈식 FastAPI 라우터(main.py -88%), 25 개 중요 + 68 개 높음 보안 finding 수정, 클래스 A Docker 관리 프록시, RBAC 사용자 관리(PBKDF2), 2 단계 시스템 업데이트(?local_only=true), AnythingLLM 프록시 통합, 휴대용 1 클릭 설치.
v0.5.0	경제 및 ERP(알파)	완전한 제 3 기둥(CRM/PocketBase 동기화, 품목/서비스, ANG/AB 내역, DATEV·GiroCode 회계, 법인 사업자), Research Lab v2, 55 개 보드룸 프레임워크.
v0.3.x	Sentinel / Foundation	7 기둥 OS 커널, 70 명 전문가 인력, 34 개 이상 Evey 플러그인, FinOps 3 단계 라우팅, 임시 코드 샌드박스, 리플레이 디버깅, PII 스크러버, 시맨틱 캐시·FTS5 하이브리드 검색.

2. 시스템 화면 및 전문 스튜디오(UI 화면)

1. 화면: Mission Control(모니터링 콕핏 및 지휘소)

문제: 중앙 실시간 콕핏이 없으면 활성 에이전트, 토큰 처리량, 일일 비용, 차단된 승인이 보이지 않습니다.

작동 원리: 5 개 핵심 KPI 타일(플릿 부하, 처리량 TPM, 비용 감시, 안정성, 승인 게이트), 5 열 보드 프로젝션, Active Process Inspection Bar(하트비트·도구 동시성·컨텍스트 포화용 3 슬롯 푸터), 15 분 토큰 활동 벨트(SVG 스파크라인), 실시간 감사 스트림을 묶습니다.

이점: *단일 화면에서 전체 운영체제의 지연 없는 360° 모니터링을 제공합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

2. 화면: 운영 – 칸반 보드 및 작업 엔진

문제: 순수 채팅 기록의 작업은 묻히고 상태·책임별로 제어할 수 없습니다.

작동 원리: kanban.db(Single Source of Truth) 기반 5 개 수명 주기 열(To-Do, 진행 중, Review 및 QA, 차단됨, 완료됨). 열별 구성 가능 WIP 한도, 우선순위 필터(P0–P3), 에이전트 할당, 시간 기록, 원클릭 Obsidian 연결을 지원합니다. 4 안 품질 게이트 통합: 작업을 'done'으로 이동 전 Peer 리뷰어(Katharina Völk, QM/준법 또는 Robert)가 작업 결과를 완전성·실체·실행 가능성으로 검사합니다(1–100% 점수). 'changes_requested' 시 상태 잠금, 필요 시 Human-in-the-Loop Override(force=True)가 우선합니다.

이점: *모든 운영 프로젝트를 투명하게 구조화하고 WIP 제한으로 시스템 과부하를 방지합니다.* 강제 4-아이 검토로 'done' 상태의 미완성 작업 상태를 방지합니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

3. 화면: 운영 – 대화형 2D 조직도 및 보고 체계

문제: 에이전트의 평면 목록은 차단된 프로세스의 지휘 권한이나 에스컬레이션 경로를 보여주지 않습니다.

작동 원리: 56 개 노드·55 개 보고선을 인터랙티브 2D 그래프로 시각화합니다(Apex -> CEO Robert -> 전문 리드 -> 전문가). 상사 변경 드래그 앤 드롭, 격자 맞춤, 팀 카드 보기, config/org_hierarchy.json 영구 저장을 지원합니다.

이점: *차단 시 시스템이 자동으로 따르는 구속력 있는 에스컬레이션·위임 체인을 정의합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

4. 화면: 운영 – 내부 에이전트 통신(무전)

문제: 전문가 간 하위 작업 전달이 사용자 채팅을 범람시키거나 보이지 않게 남았습니다.

작동 원리: 전용 도청 센터: 채널 목록(#allgemein, #tech-dev, #legal, #marketing)과 kanban.db 의 모든 에이전트 간 합의 연대순 메시지 피드.

이점: *주 사용자 받은편지함에 부담 없이 에이전트 플릿의 협업을 추적 가능하게 합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

5. 화면: 수신 및 트리아지(3 열 HITL 게이트키퍼 및 받은편지함)

문제: 에이전트의 무통제 쓰기(예: 이메일 발송)와 메일·Telegram 의 흩어진 수신은 보안 리스크를 내포합니다.

작동 원리: 3 열 triage 센터: 1 열 노란 HITL 승인(승인/거부), 2 열 IMAP/SMTP 이메일 동기화(자동 triage·위임), 3 열 미러링 실시간 Telegram 스트림(@Beatman_bot).

이점: *운영자의 명시적 승인 없이 외부 쓰기 작업이 실행되지 않도록 보장합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

6. 화면: 에이전트 플릿 및 플릿 관리(8 개 탭 콕핏)

문제: 전문가는 개별 모델, 시스템 프롬프트, 도구, 작업 지침을 한곳에 모아야 합니다.

메커니즘: 빠른 필터(활성, 즐겨찾기, 전체, 7 개 도메인), Provider·모델 핫스왑, 8 개 전문 탭으로 70 개 전문가 프로필을 관리합니다: 개요(360° 문서·Telegram 호출명), 직접 채팅, 역량(Skills), Operations(작업 기록), SOUL(시스템 프롬프트 편집기), 인사 파일(SOP), 인재 마켓플레이스, 스킬 마켓플레이스.

활용: 전체 에이전트 기반의 초 단위 보정·확장을 위한 중앙 인사부.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

7. 화면: 마스터 채팅(듀얼 모드 워크스페이스 및 멀티 세션 사이드바)

© Michael Keller

문제: 에이전트와 직접 작업 시 1:1 대화, 그룹 채팅, 미션 주문 간 빠른 전환의 집중 전체 화면 환경이 없었습니다.

작동 원리: 듀얼 모드 스위치('직접' 즉시 답변 vs. '미션' 구조화 작업 생성), 다중 파일 업로드(최대 5 개), 브라우저 화면 캡처(카메라 버튼), 음성 입력(받아쓰기), 추론 예산 선택기(0~16k 토큰), AI 프롬프트 옵티마이저, 오른쪽 세션 기록(에이전트 vs. 그룹)을 제공합니다. Telegram 음성 파이프라인 포함: 이동 중 수신 음성 메시지(.ogg/Opus)를 Whisper/Gemini 로 전사하고 호출명(/robert, /anwalt 등)으로 전문 에이전트에 라우팅하며 답변을 edge-tts(de-DE-ConradNeural) 자연 음성 메모로 Telegram 에 직접 회신합니다.

이점: *고속 대화, 멀티모달 입력, 미션 제어를 하나의 연속 작업 공간에 통합합니다.* Telegram 을 통한 완전한 모바일 음성 조작 및 양방향 오디오 교환을 가능하게 합니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

8. 화면: AI Mastermind 및 보드룸(55 개 방법론 프레임워크)

문제: 복잡한 전략 결정에는 개별 의견이 아닌 구조화된 경영 방법론과 논쟁적 관점이 필요합니다.

메커니즘: 7 개 카테고리 34 개 선택 프레임워크(35 개 Base, 예: CLEAR, Design Sprint, Double Diamond, Lean Startup, First Principles, Game Theory, Blue Ocean, Pre-Mortem, Six Sigma)로 다단계 전체 회의(Chairman + 3 개 실제 전문 에이전트 슬롯 + 레드팀 챌린저)를 시뮬레이션합니다.

활용: 방법론적 엄격성을 강제하고 레드팀으로 맹점을 드러내며 결정-ready 보드 메모를 제공합니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

9. 보기: Showroom (프로젝트 탐색기, 아티팩트 갤러리 및 Python 샌드박스)

문제: 생성된 보고서, HTML 프로토타입, 계산서 및 스크립트는 긴 대화에서 손실되었습니다.

작동 원리: 생성자, 날짜, 유형별 모든 작업 결과를 목록화합니다. Markdown/HTML/JSON 실시간 렌더링의 Workspace Vault Reader 와 안전한 코드 실행용 격리 Python 샌드박스를 포함합니다. 인터랙티브 Showroom Live-Runner 로 확장: 12 초 타임아웃, 격리 스크래치 디렉터리, 펼침 터미널 서랍(STDOUT, STDERR, 종료 코드, 밀리초 실행 시간)의 브라우저 직접 Python·코드 파일 실행을 지원합니다.

이점: *고객·프로덕션 투입 전 모든 완성 deliverable 의 인수·발표 홀 역할을 합니다.* 코드 아티팩트와 스크립트는 콘솔 전환 없이 Showroom 에서 직접 검증/테스트할 수 있습니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

10. 보기: 중앙 라이브러리 및 2nd Brain (RAG, 자동 OCR 및 Obsidian-Vault)

문제: PDF 전문 서적, DIN 규격, 건축 평면도는 수동 복사 없이 에이전트가 검색할 수 없었습니다.

메커니즘: 3 개 하위 탭·4 개 Core Hub(00_HR_Agenten, 02_Wissensbibliothek, Protokolle/, 00_Notebook_Synthesen) 통합: 1. AnythingLLM RAG 인덱스(Port 3001), 2. 드래그 앤 드롭·자동 OCR(Docling)·AI

주제 분류 문서 수집기, 3. 직접 Obsidian 파일 트리·Markdown 뷰어(Vault 전환 포함).
활용: PDF 감정서를 모든 70 개 에이전트용 검색 가능 Markdown·벡터 지식으로 완전 자동 변환합니다.
비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

11. 화면: 워크플로 엔진(포트 5678 의 n8n Automation Mesh)

문제: 반복 인터페이스 프로세스(Cron 작업, 웹훅, DB 동기화)는 LLM 으로 실행하면 토큰을 불필요하게 소모합니다.

작동 원리: 시각적 n8n 워크플로 편집기를 직접 내장하고 FastMCP 스킬 n8n-automation-mesh 로 CEO 에이전트 Robert 와 연결합니다. 활성화된 0 토큰 루틴: 1. Morning-Briefing(평일 07:30, Obsidian 저장·Telegram 전달), 2. Weekly Maintenance(일요일 23:00, SQLite 백업·상태 감사), 3. RSS·News Watchdog(시간별 스캔), 활성 5 개 기능 웹훅(웹 추출, 메시징, 문서 렌더링, 피드 검색, CRM 동기화).

이점: *결정적 0 토큰 백그라운드 프로세스와 핵심 지점의 인지 에이전트 결정을 결합합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

12. 화면: LLM 게이트웨이(OmniRoute AI 라우터 대시보드)

문제: 단일 클라우드 공급자에 고정되면 속도 제한·장애 시 시스템이 즉시 정지합니다.

작동 원리: 중앙 프록시 대시보드(Port 20128 / 8000): 다중 공급자 Vault, 실시간 지연 측정(ms), HTTP 오류 모니터(429/500), 자동 서킷 브레이커, 4 단계 폴백 캐스케이드(GLM-4.7 Flash -> Gemini 2.5 Flash -> Claude 3.5 Sonnet -> 로컬 Llama 3.3 70B).

이점: *최소 API 비용으로 모든 인지 프로세스의 99.9% 가용성을 보장합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

13. 화면: Token Guard(3 개 하위 화면의 Guardian 및 토큰 분석)

문제: 어떤 에이전트·모델·개별 세션이 가장 많은 토큰·비용을 유발하는지 투명성이 부족했습니다.

작동 원리: 시간 필터(24h, 7d, 30d, 전체), 비활성 타이머, 3 개 하위 뷰 제공: 1. 개요·일일 추세 차트(프로필/모델 테이블, 보호 장벽 양식), 2. 개별 세션 소비 검사기(입력, 출력, Cache-Read 토큰 정렬), 3. 자원·작업 스로틀(슬라이더 1–16 작업).

이점: *개별 세션 ID 까지 완전한 비용 통제와 즉시 예산·병렬 제한을 포함합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

14. 보기: 확장 프로그램 및 모듈 센터 (34 개 Evey 전용 플러그인)

문제: 보안·저장·분석 기능을 전문 분야별로 모듈식으로 켜고 끌 수 없었습니다.

작동 원리: 8 개 전문 탭 경유 34 개 전문 Hermes/Evey 플러그인 조종: 보안·가드(4), 성능·캐시(2),

모니터링·분석(3), 에이전트 지능(6), 계획·작업(2), 브리지·인터페이스(4), 기억·지식(3), 도구·실행(3), 금융/품질/유지보수. 표준 플러그인 시스템·런타임 훅 포함: 모듈식 폴더 구조(plugins/<plugin_id>/plugin.json), 4 개 수명 주기 훅(on_load, on_enable, on_disable, on_unload), 2 개 도구 인터셉터(before_tool_execute, after_tool_execute), 사전 설치 스타터 플러그인(System Metrics, Security Audit Hook, Cost Guard).

이점: *운영 중 마우스 클릭으로 개별 시스템 기능을 정밀하게 켜고 끌 수 있습니다.* 커널 코드 변경 없이 격리된 런타임 확장과 전역 도구 인터셉션을 가능하게 합니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

15. 화면: Research Lab(심층 검색 및 근거 삼각측량)

문제: 단순 검색 쿼리는 출처 대조나 신뢰도 평가 없는 미검증 단일 결과를 제공합니다.

작동 원리: 선택 탐색 깊이, 출처 모니터, 근거 점수, 상충 주장 삼각 측량 실시간 종합 프로토콜, Obsidian-Vault 원클릭 내보내기의 다단계 리서치 콕핏. 자율 심층 조사·성찰 루프 v2(Re-Search Loop) 포함: 복잡 조사를 3–5 개 하위 질문으로 분해, 자율 격차 분석, 최대 3 회 반복 후속 검색, 다중 공급자 폴백(SearXNG / Jina Reader), SSRF 보호.

이점: *신뢰할 수 있고 인용 가능한 시장·법률·기술 문서를 생성합니다.* 모순·불완전 출처에 대한 재귀 후속 조사로 지식 격차를 스스로 메웁니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

16. 화면: 텔레메트리 및 로그(3 개 소스 라이브 감사 스트림)

문제: 컨테이너나 에이전트 프롬프트 오류 발생 시 지금까지 터미널에서 Docker 명령으로 힘들게 찾아야 했습니다.

작동 원리: 3 개 로그 소스 실시간 묶음: 물리적 Docker 컨테이너 로그(11 개 컨테이너 stdout/stderr), 인지 에이전트 로그, 암호화 감사 추적. 레벨 필터(DEBUG, INFO, WARN, ERROR), 자동 스크롤 잠금, 내보내기 포함.

이점: *초 단위 오류 진단 및 컴플라이언스 증빙용 변조 불가 블랙박스.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

17. 특별 스튜디오: Archon Prompt Studio(832 개 템플릿 및 Fabric SOP)

문제: 부정확한 작업 지시는 환각, 잘못된 출력 형식, 고비용 수정 루프를 유발합니다.

메커니즘: 16 개 전문 카테고리 824 개 엔터프라이즈 템플릿, 255 개 이상 Fabric-SOP 패턴, 5 필드 공식(1. 역할, 2. 컨텍스트, 3. 과제, 4. 형식, 5. 제약/부정 규칙), AI 프롬프트 옵티마이저의 통합 빌딩 블록.

활용: 모든 위임을 명확한 구조로 강제하고 클릭으로 채팅·Kanban 작업에 전달합니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

18. 특별 스튜디오: Visual CSS Live Studio 및 인플레이스 변환 기즈모(Alt + S)

문제: 디자인 조정에는 지금까지 CSS 파일 검색과 브라우저 지속 새로고침이 필요했습니다.

작동 원리: 부유 실시간 디자인 스튜디오(Alt + S): 크로스헤어 DOM 인스펙터, 인터랙티브 변환 기즈모(위치 드래그, 4 모서리/가장자리 스케일, 모서리 둥글기 드래그), 테마 프로필 관리자, custom_studio.css 직접 내보내기.

이점: *페이지 새로고침 없이 운영 중 전체 UI 의 픽셀 단위 실시간 스타일링을 지원합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

19. 특별 스튜디오: Design System Lab, AppShell 아키텍트 및 15 개 컴포넌트 스튜디오

문제: 라이트/다크 모드 간 상이한 헤더 높이, 간격 또는 대비가 시각적 안정감을 해칩니다.

작동 원리: 인터랙티브 AppShell 아키텍트(56px 헤더 여백, 사이드바 너비, 뷰포트 패딩 실시간 축소 시뮬레이션)와 15 컴포넌트 디자인 스튜디오(os-workspace~os-badge), 듀얼 테마 엔진(Light/Dark)을 포함합니다.

이점: *20 개 모든 화면에서 일관되고 눈부심 없는 B2B SaaS 외관을 보장합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

20. 특별 화면: System Control Center(11 개 전문 탭, 라이브 독) 및 전역 옴니박스(Ctrl + K)

문제: 설정이 분산되어 있었고 20 개 보기·70 개 에이전트 탐색에 많은 클릭이 필요했습니다.

메커니즘: System Control Center 는 7 개 관리 탭(Hermes Core(YAML 편집기 포함), 컨테이너·서비스, Provider·Gateway, 방법론, 플러그인, 통신, 디자인)을 묶고 'Live-Dock'으로 오른쪽에 도킹할 수 있습니다. Omnibox(Strg + K)가 키보드로 모든 탭(#), 에이전트(@), 명령에 즉시 이동합니다. 원클릭 업데이트 시스템·지속 배포 통합: 모든 SQLite DB 의 사전 업데이트 스냅샷(WAL, data/snapshots/) 포함 자동 GitHub 동기화(매 Git-Pull 전), 원클릭 롤백, 마운트 타임아웃 대비 최소 Git-Stat 검사를 통합합니다.

활용: 관리자·파워 유저의 최대 조작 속도. 자동 DB 롤백 포함 안전한 SW 업데이트를 지원합니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

3. 토큰 절감 및 FinOps 생태계(비용 최소화)

21. 3 계층 자동 비용 라우팅 상한(LOW / MEDIUM / HIGH)

문제: 단순 루틴 작업이 고가 최상위 모델(Claude Sonnet, GPT-4o 등)에서 실행되면 일일 비용이 폭증합니다.

작동 원리: System Control Center 의 100 만 토큰당 구성 가능 가격 상한: LOW(최대 $0.10/1M -> Free-Tier/Flash-Lite 강제), MEDIUM(최대 $1.00/1M -> Haiku/Gemini Flash), HIGH(상한 없음 -> 복잡 아키텍처용 최고 추론 모델).

이점: *어려운 작업의 품질 손실 없이 일상 API 지출을 최대 85% 까지 절감합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

22. FreeLLM API Docker 프록시 및 'auto/best-free' 라우팅(포트 3002)

문제: 다양한 공급자의 무료 할당량은 수동 모니터링이 어렵고 자주 변경됩니다.

작동 원리: 전용 Docker 컨테이너(freellmapi, Port 3002)가 무료 LLM 엔드포인트를 묶습니다. 'auto/best-free' 라우팅 프로필이 가장 강력한 0 유로 모델을 자동 선택합니다.

이점: *정확히 $0.00 API 비용으로 백그라운드 및 트리아지 에이전트의 지속 운영을 가능하게 합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

23. 마스터 채팅 및 라우터의 무단계 추론 예산 제어(0~16k 토큰)

문제: 추론 모델(Thinking 모드)은 사소한 질문에도 수천 개의 보이지 않는 사고 토큰을 소모합니다.

작동 원리: 채팅 푸터와 agent_router.py 의 6 단계 조절기(끄기: 0Tok, 최소: ~1k, 낮음: ~2k, 중간: ~4k, 높음: ~8k, 최대: ~16k).

이점: *단순 작업에서 고비용 사고 루프를 완전히 끄고 진짜 복잡한 경우에 심층 추론을 예약합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

24. Evey 캐시 및 캐시 TTL 제어(24 시간 시맨틱 및 정확 응답 캐시)

문제: 반복되는 동일·유사 시스템 조회가 매번 새로운 유료 API 호출을 발생시켰습니다.

작동 원리: evey-cache 플러그인은 구성 가능한 수명(기본 24 시간 TTL)으로 모델 응답을 저장하고 세션 인스펙터에 절약된 Cache-Read 토큰을 별도 표시합니다.

이점: *반복 조회에 0 밀리초, 0 토큰 비용으로 응답합니다.*

비용과 그림자(트레이드오프): 캐시 TTL 이 너무 높으면(> 72 시간) 동적 주제에서 오래된 답변을 제공합니다.

언제 비활성화합니까? 및 임곗값: 실시간 조회(실시간 시세, 서버 상태) 시 비활성화. 기본 TTL: 24 시간.

25. Guardian 의 Max-Context 및 Max-History 하드 클리퍼 (32k / 20 턴)

문제: 끝없이 커지는 채팅 세션은 새 질문마다 전체 기록을 재전송해 입력 토큰을 기하급수적으로 늘립니다.

작동 원리: Guardian 에서 최대 컨텍스트 창(예: 32,000 토큰)과 전달 최대 채팅 메시지 수(예: 최근 20 턴)의 하드 제한을 설정합니다.

이점: *턴당 최대 입력 비용을 위쪽으로 안정적으로 제한합니다.*

비용과 그림자(트레이드오프): 32k 토큰 또는 20 턴을 초과하는 메시지를 하드 절단하여 아주 이른 컨텍스트가 사라집니다.

언제 비활성화합니까? 및 임곗값: 심층 문서 분석을 위해 Hermes Core 에서 64k–128k 로 상향.

26. 로우 컨텍스트 모델용 동적 토큰 클리퍼(backend/token_clipper.py)

문제: 대규모 도구 출력이 컨텍스트 창을 초과하면 소형/로컬 모델(8k/16k)이 중단됩니다.

작동 원리: 시스템 프롬프트(인덱스 0)와 마지막 사용자 지침을 반드시 보존하고, 과장된 개별 블록을 지능 축소하며 오래된 중간 단계를 제거합니다.

이점: *대규모 작업에서도 극히 저렴한 소형 모델의 안정적 사용을 허용합니다.*

비용과 그림자(트레이드오프): 과장된 도구 출력을 중간에서 잘라내고([...SNIP...]) 원시 텍스트의 개별 코드 줄이 누락될 수 있습니다.

언제 비활성화합니까? 및 임곗값: 전체 원시 로그가 필요할 때 일시적으로 비활성화하십시오.

27. 슬라이딩 윈도 메모리 압축 (통합된 결정 컨텍스트)

문제: 오래된 메시지를 단순 절단하면 이전 프로젝트 결정이 손실됩니다.

작동 원리: 오래된 대화 라운드를 간결 요약으로 압축해 [통합 대화·결정 컨텍스트] 블록으로 1 번 위치에 주입합니다.

이점: *내려진 모든 결정을 완전히 유지하면서 과거 대화 토큰을 최대 70% 절감합니다.*

비용과 그림자(트레이드오프): 압축 단계는 일회성으로 약 800 토큰이 소모되며 초기 라운드의 문구를 다듬습니다.

언제 비활성화합니까? 및 임곗값: 필요 시 Hermes Core 의 protect_last_n(기본값: 20)으로 미세 조정하십시오.

28. 서브에이전트 RAM 용 LRU-K 페이지 스와핑 (backend/context_swapper.py)

문제: 병렬 서브에이전트가 방대한 원시 데이터를 활성 프롬프트 메모리에 동시 유지합니다.

작동 원리: 80% 사용률부터 급하지 않은 컨텍스트 페이지를 LRU 원칙으로 SQLite 에 오프로드하고 필요 시만(rehydration) 다시 로드합니다.

이점: *프롬프트를 간결하게 유지하고 비활성 배경 지식에 대한 불필요한 토큰 비용을 방지합니다.*

비용과 그림자(트레이드오프): 에이전트 RAM 의 디스크 스왑 아웃/로드 시 컨텍스트 전환에 약 10–25ms I/O 지연 발생.

언제 비활성화합니까? 및 임곗값: OOM 충돌 방지를 위해 RAM 사용률 80%부터 활성 상태로 유지.

29. JEV 라우터 — 0 토큰 결정적 인텐트 사전 필터링 (jev_router.py)

문제: 단순 시스템 지시마다 대형 언어 모델로 분류하는 것은 토큰과 시간 낭비입니다.

작동 원리: LLM 호출 전 결정적 JSON Execution Vector 와 키워드 휴리스틱으로 수신 입력을 분류합니다.

이점: *단 하나의 LLM 토큰도 소모하지 않고 명확한 시스템 명령을 대상 루틴에 직접 전달합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

30. ASD-STE100 간소 기술 영어 프롬프터(.agents/skills/asd-ste100-prompter)

문제: 시스템 프롬프트와 에이전트 지침의 장황한 산문은 모든 API 호출을 불필요하게 부풀립니다.

작동 원리: 항공 표준 ASD-STE100(단어당 하나의 의미, 능동 동사, 짧은 문장, 무충전어)으로 지침을 변환합니다.

이점: *시스템 프롬프트 토큰 길이를 약 30–40% 줄이고 동시에 도구 호출 오류율을 낮춥니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

31. Anti-AI-Slop 및 출력 압축 (.agents/skills/anti-ai-slop)

문제: 언어 모델은 기본적으로 긴 정중 상투어, 서론, 중복 요약을 생성합니다(고비용 출력 토큰).

작동 원리: 시스템 프롬프트에서 AI 상투어, 빈 문구, 인위적 부풀리기를 차단하고 출력을 정제합니다.

이점: *특히 비싼 출력 토큰을 최대 25% 절감하고 즉시 활용 가능한 전문 텍스트를 제공합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

32. 사전 비행 비용 추정기 및 예산 예측 (backend/cost_estimator.py)

문제: 대규모 작업 시작 전 실행 비용을 예측할 수 없었습니다.

작동 원리: POST /api/router/estimate-cost 로 현재 모델 가격표 기준 예상 입력/출력 토큰·달러 금액을 사전 계산해 작업 예산과 대조합니다.

이점: *첫 1 센트가 지출되기 전에 비용이 큰 실행을 경고합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

33. 동적 작업 예산(80% 절감 다운그레이드 및 100% 중단)

문제: 하나의 복잡한 작업이 눈에 띄지 않게 일일 예산 전체를 소모할 수 있었습니다.

작동 원리: 작업당 누적 비용(예: 한도 $0.25) 모니터링: 80%에서 저렴한 절약 모델로 자동 전환하고 100%에서 Telegram 승인 요청과 함께 작업을 일시정지합니다.

이점: *다단계 자율 리서치에서 비용 폭주를 방지합니다.*

비용과 그림자(트레이드오프): 작업 예산 80% 도달 시 저렴한 절약 모델로 다운시프트하고 100%에서 하드 스톱합니다.

언제 비활성화합니까? 및 임곗값: 대규모 아키텍처 실행용 작업 예산(기본값: 0.50$)을 2.00$로 인상하십시오.

34. Loop-Killer 및 최대 단계 제한(15 단계 / Jaccard > 90%)

문제: 에이전트가 결함 있는 도구 호출에서 무한 루프에 빠지면 초 단위로 잔액을 소모합니다.

작동 원리: 각 단계를 정확한 반복(>= 3)이나 의미 순환 참조(> 90% 유사)로 검사하고 설정 가능한 Max-Steps 한도(기본 15 단계) 이후 작업을 중단합니다.

이점: *밤사이 눈에 띄지 않는 무한 루프로부터 계정을 안정적으로 보호합니다.*

비용과 그림자(트레이드오프): 너무 날카로운 임계값(동일 최대 = 2 또는 Jaccard < 0.80)은 단계적 테스트 수정·페이지네이션 같은 정당한 반복 작업을 너무 일찍 중단시킵니다.

언제 비활성화합니까? 및 임곗값: 에이전트가 의도적으로 많은 동형 루프 단계(예: 50 개 파일 대량 마이그레이션)를 수행해야 하면 잠시 끄십시오. | 임계값: 최대 동일: 2 = 재시도 중단 | > 5 = 고비용 루프 (기본값: 3). Jaccard: < 0.80 = 오탐 | > 0.96 = 늦은 감지 (기본값: 0.90). 최대 단계: 기본값 15.

4. 커널, 스케줄링 및 데몬 아키텍처(Pillar 1)

35. Hermes Master Orchestrator (Hermes-Core)

문제: 조정되지 않은 개별 에이전트가 서로 대립하거나 전체 목표를 놓칩니다.

작동 원리: 작업 분석, 역할 선택, 모니터링 및 결과 인수용 중앙 프로세스 지휘소.

이점: 70 명 스페셜리스트 전체의 전략적 일관성을 보장합니다.

가격 및 단점(Trade-off): 활성 백그라운드 서비스와 높은 추론('high')은 기본 부하·토큰 비용을 높입니다.

언제 끕니까? 및 임곗값: 'HERMES CORE' 탭의 14 개 섹션을 통한 세밀한 제어.

36. Tickless 커널 데몬 및 Asyncio 이벤트 루프(포트 8420 / 8501)

문제: 초 단위 지속 폴링은 CPU 리소스와 시스템 에너지를 낭비합니다.

작동 원리: 다음 만기 작업의 정확한 시점을 계산('Sleep-until-next-due')하고 그때까지 커널을 휴면 상태로 전환합니다.

이점: *유휴 시 최소 시스템 부하와 지연 없는 대기 해제 반응.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

37. 우선순위 선점 스케줄러(P0–P3 대기열)

문제: 백그라운드 작업이 모든 슬롯을 차지하면 직접 채팅의 긴급 사용자 질문이 대기해야 했습니다.

작동 원리: 우선순위 큐: P3 작업(직접 채팅/긴급)이 실행 중인 P1/P2 루틴 작업을 즉시 추월합니다.

이점: *높은 백그라운드 부하에도 대화에서 빠른 응답 시간을 보장합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

38. 자유롭게 확장 가능한 동시성 스로틀링 (backend/concurrency_throttle.py)

문제: 과도한 동시 에이전트는 HTTP-429 속도 제한과 99% CPU 피크를 유발합니다.

작동 원리: Asyncio 세마포어로 활성 워커를 조절합니다(Guardian·Control Center 의 슬라이더로 1~16 병렬 작업 설정).

이점: *노트북의 스로틀 보호 운영과 워크스테이션의 고부하 운영을 모두 허용합니다.*

비용과 그림자(트레이드오프): 너무 빡빡한 정체 한도(< 30 초)는 느린 심층 추론 모델(예: o3/R1)을 사고 중에 잘못 중단시키고, 워커 증가는 RAM·API 피크 부하를 높입니다.

언제 비활성화합니까? 및 임곗값: 매우 긴 단일 분석·느린 하드웨어의 로컬 LLM 실행 시 끄십시오. | 임계값: 최대 워커: 1–2 = 적체 | > 8 = RAM 피크·HTTP-429 (기본값: 4). 정체: < 30 초 = 사고자 중단 | > 180 초 = 늦은 치유 (기본값: 45 초).

39. 자율 30 초 워치독 및 충돌 복구 (watchdog_daemon.py)

문제: 충돌하거나 멈춘 작업이 in_progress 상태에 영구히 고착되었습니다.

작동 원리: 30 초마다 모든 실행 프로세스를 검사하고, 타임아웃(120 초) 후 좀비 작업을 종료하며 고아 작업을 자동 충돌 복구로 넘깁니다.

이점: *모든 워커 슬롯에 좀비 항목이 없도록 유지합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

40. 교착 상태 및 정체 수퍼바이저 (backend/deadlock_supervisor.py)

문제: 두 에이전트 간의 상호 의존성으로 조용한 멈춤이 발생할 수 있습니다.

작동 원리: 순환 대기 상태와 진전 없는 작업(> 45 초)을 감지해 지수 백오프로 자율 해소합니다.

이점: *복잡한 그룹 작업 시 파이프라인 정지를 방지합니다.*

비용과 그림자(트레이드오프): 너무 빡빡한 정체 한도(< 30 초)는 느린 심층 추론 모델(o3/R1)을 사고 중에 중단시킵니다.

언제 비활성화합니까? 및 임곗값: 추론 모델 사용 시 정체 한도를 90–120 초로 상향(기본값: 45 초).

41. 플릿 자가 치유 및 컨테이너 자동 재시작(backend/fleet_self_healing.py)

문제: 11 개 Docker 컨테이너 중 하나라도 중단되면 수동으로 재시작해야 했습니다.

작동 원리: 보안 docker-socket-proxy 경유 모든 Core 컨테이너를 모니터링하고 장애 시(> 60 초) 감사 항목 포함 자율 재시작합니다.

이점: *전체 Docker 환경의 무인 24/7 운영을 보장합니다.*

비용과 그림자(트레이드오프): 영구 결함 컨테이너는 자동 재시작으로 재시작 루프에 빠질 수 있습니다.

언제 비활성화합니까? 및 임곗값: 수동 컨테이너 유지보수 작업 중에는 일시적으로 비활성화하십시오.

42. 선제적 아침 스탠드업 및 기상 서비스 (backend/autonomous_standup.py)

문제: 아침에 야간 실행, 비용 및 대기 중인 결정에 대한 간결한 개요가 없었습니다.

작동 원리: 매일 08:00 Obsidian-Vault(01_PROJEKTE/Daily_Briefings/)에 통합 상태 보고서를 생성해 대시보드·Telegram 에 푸시합니다.

이점: *완성된 의사결정 템플릿으로 매 업무일을 시작합니다.*

비용과 그림자(트레이드오프): 각 자동 아침 브리핑은 설정된 시점에 약 1,500–3,000 토큰을 소모합니다.

언제 비활성화합니까? 및 임곗값: 일일 아침 보고서가 필요 없으면 비활성화(기본값: 08:00).

5. 멀티 에이전트 프로세스 모델, 계층 구조 및 통합(Pillar 2)

43. 70 개 선언적 에이전트 프로필 및 Markdown 인사 기록

문제: 일반 AI 비서는 역할 인식과 정의된 품질 기준이 없습니다.

메커니즘: 70 명 전문가는 agent_profiles.json 과 자체 인사 파일(00_HR_Agenten/)에 시스템 프롬프트, 모델 경로, 도구 범위, SOP 와 함께 고정됩니다.

이점: 전략, 법률, 건설, 재무, 마케팅 및 IT 용 신뢰할 수 있는 가상 전문가를 만듭니다.
비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

44. Robert 위임, Fan-in 동기화 및 Lisa 마커 계약(SYNTHESE-BEREIT)

문제: 병렬 하위 작업에서 부분 결과가 손실되거나 불완전하게 요약되었습니다.

작동 원리: Robert 가 대형 프로젝트를 분해하고 프로세스 락(_MC_DATA_LOCK)이 모든 하위 작업 완료를 대기합니다(Fan-in). Lisa 가 SYNTHESE-BEREIT 필수 마커로 보고서를 집계하고, Robert 가 1 행에 정식 승인을 부여합니다.

이점: *실제 기업 경영을 본뜬 다단계 품질 보증.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

45. 표준화된 에이전트 간 핸드오프 데스크 (backend/handoff_desk.py)

문제: 한 스페셜리스트에서 다음으로 인계 시 조용한 컨텍스트 손실.

작동 원리: 납품물, 미결 가정, SHA-256 체크섬의 정식 인수 프로토콜을 생성하며 수신 에이전트가 수령 확인해야 합니다.

이점: *모든 부서 간 감사 대응 인터페이스.*

비용과 그림자(트레이드오프): min_content_length 가 너무 높으면(> 300) 엄격한 SLA 검사가 짧지만 정확한 팩트 답변을 거부합니다.

언제 비활성화합니까? 및 임곗값: 기본값: 최소 길이 100 자, 처리 시간 60 분.

46. 병렬 프로젝트 실행용 2 단계 DAG 스케줄러(backend/dag_scheduler.py)

문제: 대규모 프로젝트에서 많은 하위 작업의 순차 처리에 시간이 오래 걸립니다.

작동 원리: 방향성 비순환 그래프(DAG)를 계산합니다: 독립 조사(1 단계)를 병렬 실행하고 이후 종합(2 단계)을 자동 시작합니다.

이점: *복잡한 분석의 처리 시간을 수 배 단축합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

47. Self-Correction & Tool Execution Reflection Loop

문제: 도구 호출 시 일시적 구문/매개변수 오류로 즉시 중단되었습니다.

작동 원리: 도구 오류를 잡아 오류 메시지를 성찰하고 최대 3 회 시도 내 호출 매개변수를 자율 수정합니다.

이점: *파일·데이터베이스·웹 작업에서 훨씬 높은 안정성을 제공합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

48. SQLite 상태 스냅샷을 적용한 중단 가능한 장기 워크플로

문제: 작업이 사용자 승인을 수시간 기다리면 메모리를 불필요하게 점유하거나 재시작 시 상태를 잃습니다.

작동 원리: workflow_checkpoints 테이블에 정확한 실행 상태를 동결하고 승인 후 작업을 끊김 없이 재개합니다.

이점: *재시작 후에도 수일에 걸친 비즈니스 프로세스를 가능하게 합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

49. 듀얼 에이전트 합의 게이트 및 적대적 비평가 검토(backend/consensus_gate.py)

문제: 단일 에이전트는 중요 작업에서 자신의 사고 오류나 환각을 종종 알아차리지 못합니다.

작동 원리: P3 작업·중요 개입에는 독립 리뷰어 에이전트(레드팀/Critic)의 형식적 교차 검토를 강제합니다. Kanban 보드의 자동 4 안 품질 게이트로 연결됩니다(Katharina Völk / Robert 리뷰, 1–100% 점수, 수정 필요 시 상태 잠금).

이점: *모든 업무상 중요한 결과에 대한 4-아이 원칙.*

비용과 그림자(트레이드오프): 토큰 소비를 3 배 증가시키고 응답 시간을 +6~20 초 늘립니다. 만장일치 강제는 교착 위험이 있습니다.

언제 비활성화합니까? 및 임곗값: 우선순위 P3/P4 부터 활성화하고 2/3 다수결을 선택합니다.

6. MCP 드라이버, 외부 서비스 및 멀티모달 도구(Pillar 3)

50. Archon CRM 및 PocketBase 생태계(포트 3005, 8090, 8092)

문제: 고객 연락처, 이메일 기록 및 미디어 자산이 AI 에이전트와 분리되어 있었습니다.

작동 원리: 전용 CRM 프론트엔드(Port 3005), PocketBase 백엔드(Port 8090), 미디어 서버(Port 8092)를 scripts/archon_crm_mcp.py 로 완전 연동합니다.

이점: *에이전트는 연락처를 조회, 추가 조사, 평가(리드 스코어링) 및 서신 매핑할 수 있습니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

51. 자동 이메일 매칭 및 LinkedIn 아카이브 임포터

문제: LinkedIn 내보내기 및 이메일 받은편지함의 수천 개 연락처를 수동으로 대조해야 했습니다.

작동 원리: 사서함 동기화(--fetch-mails), 기존 CRM 연락처 자동 매핑, LinkedIn Connections CSV 전체 가져오기(중복 제거 포함).

이점: *수분 만에 완전하고 보강된 관계 데이터베이스를 구축합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

52. Robert 도구 권한 관리자 (backend/tool_permission_manager.py)

문제: 에이전트는 누락된 Python/Node 패키지로 무력하게 중단하지 않아야 하지만 임의 설치도 금지됩니다.

작동 원리: Robert 가 명령·근거 포함 정식 설치 요청(pending_permission)을 제출하며, 원클릭 승인 후에만 패키지가 설치됩니다.

이점: *호스트 시스템을 완전히 보호하면서 통제된 확장성.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

53. Event-Driven Webhook-Mesh (backend/event_mesh.py)

문제: GitHub, 양식 또는 타사 시스템의 외부 이벤트는 에이전트 조치를 트리거할 수 없었습니다.

작동 원리: 외부 JSON 페이로드를 검증해 실행 가능한 Kanban 작업으로 직접 변환하는 범용 웹훅 수신기.

이점: *Archon OS 를 나머지 IT 환경과 실시간으로 연결합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

54. 중앙 멀티채널 알림 허브 (backend/notification_hub.py)

문제: 중요 경고는 UI 에 묻히고, 사소한 알림이 휴대폰을 방해했습니다.

작동 원리: 심각도별 엄격 라우팅: INFO -> UI 토스트, WARNING -> 대시보드 로그, CRITICAL -> 즉시 Telegram 푸시.

이점: *알림 피로 없는 정밀한 알림.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

55. 멀티모달 비전 및 화면 캡처 파이프라인(backend/vision_pipeline.py)

문제: 이미지의 시각적 문제나 표를 힘들게 설명해야 했습니다.

작동 원리: 브라우저 내 직접 화면 캡처(채팅의 카메라 버튼)와 업로드 이미지(.png, .jpg, .webp) 자동 OCR·구조 분석.

이점: *에이전트는 화면에 보이는 그대로를 봅니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

56. Apollo 음성 엔진 — 엣지 TTS 및 Whisper 받아쓰기 (apollo_voice_engine.py)

문제: 스마트폰/데스크톱 장문 입력은 시간이 소모되며, 보고서는 이동 중 듣고 싶어 합니다.

작동 원리: 양방향 음성 파이프라인: 채팅 입력란 직접 마이크 받아쓰기(Speech-to-Text)와 브리핑용 자연 음성 출력(Text-to-Speech). 완전한 Telegram 음성 통합(POST /api/telegram/voice-relay, POST /api/voice/synthesize)으로 edge-tts(de-DE-ConradNeural) 경유 모바일 음성 메모.

이점: *Archon OS 를 음성 제어 임원 비서로 전환합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

57. Oracle News Radar & SEO Execution Agent

문제: 시장 변화와 자사 웹사이트의 SEO 취약점은 산발적으로만 점검되었습니다.

작동 원리: Oracle News Radar 가 프로젝트 관련 전문 출처를 지속 스캔하고, SEO 에이전트가 메타 태그·구조·키워드 밀도를 검사합니다.

이점: *백그라운드에서 지속적인 시장 및 가시성 최적화.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

7. 기억, 데이터베이스 위생 및 Obsidian 2nd Brain(Pillar 4)

58. 중앙 SQLite WAL 메모리 계층(5 개 기본 테이블)

문제: 여러 에이전트의 동시 읽기/쓰기 액세스는 단순 파일에서 충돌을 유발했습니다.

작동 원리: 트랜잭션 SQLite DB(data/agentic_memory.db), WAL 모드, 5 개 핵심 테이블 tasks, conversations, learnings, kv_store, user_profile.

이점: *최고의 데이터 보안으로 블로킹 없는 병렬 액세스.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

59. 원자적 파일 교환의 프로세스 안전 칸반 사이드카(backend/kanban_sidecar.py)

문제: WIP 한도나 태그 같은 메타데이터가 병렬 쓰기 시 손상될 수 있었습니다.

작동 원리: 프로세스 전역 뮤텍스 및 원자적 쓰기 교체 연산으로 config/kanban_meta.json 을 보호합니다.

이점: *고빈도 보드 업데이트 시 데이터 손실을 방지합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

60. AnythingLLM 용 Docker-RAG 브리지(포트 3001 에 5,068 개 문서 인덱싱)

문제: 일반 채팅 요청 시 Docker 벡터 저장소의 지식이 자동으로 참조되지 않는 경우가 많았습니다.

작동 원리: backend/rag_bridge.py 가 에이전트 코어를 AnythingLLM 워크스페이스 ARCHON 에 직접 연결하고 Grounded Context 로 관련 텍스트를 제공합니다.

이점: *일반적 추정이 아닌 실제 기업 문서 기반의 근거 있는 답변.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

61. Obsidian 볼트 내 3 단계 프로젝트 폴더 배치

문제: 에이전트가 파일을 생성할 때 고정 규칙이 없으면 볼트가 금세 어지러워집니다.

작동 원리: 각 프로젝트에 자동으로 3 단계 구조를 생성합니다: 01_중간보고, 02_미디어, 03_최종산물.

이점: 원시 초안과 승인된 최종 결과를 깔끔하게 분리합니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

62. Obsidian 볼트 인덱서 및 자동 콘텐츠 맵(backend/vault_hygiene.py)

문제: 새 프로젝트 보고서가 Obsidian 그래프에서 종종 연결되지 않은 채 남았습니다('고아 노트').

작동 원리: Vault 를 스캔해 고아 노트를 찾고 중앙 _MOC_Projekt_Index.md 를 위키링크로 완전 자동 업데이트합니다.

이점: Obsidian 의 지식 그래프를 항상 완전히 연결된 상태로 유지합니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

63. 자율 팩트 수집 및 85% 유사도 중복 제거(backend/post_task_pipeline.py)

문제: 완료된 작업의 중요 인사이트가 사라지거나 여러 번 중복 저장되었습니다.

작동 원리: 작업 완료 후 learnings·user_profile 용 새 사실을 자동 추출하고 85% 이상 유사 시 중복을 방지합니다.

이점: 시스템은 메모리를 어지럽히지 않으면서 완료된 작업마다 더 스마트해집니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

64. 7 일 순환 자동 온라인 DB 백업(backend/db_backup_manager.py)

문제: 정기 DB 백업 없이는 하드웨어 결함 시 모든 작업과 프로필 손실 위험이 있습니다.

작동 원리: 운영 중 PRAGMA wal_checkpoint(TRUNCATE)을 실행하고 압축 .db.gz 아카이브를 .hermes/backups/db/에 저장합니다.

이점: 1 초의 다운타임 없는 빈틈없는 데이터 백업.

비용과 그림자(트레이드오프): 일일 전체 백업은 보관 기간에 따라 약 50–500MB 디스크 공간을 차지합니다.

언제 비활성화합니까? 및 임곗값: 기본값: 7 일 보관(db_backup_keep_days).

65. 자율 SQLite Vacuum 및 단편화 최적화기(backend/db_optimizer.py)

문제: 삭제된 레코드와 잦은 업데이트로 SQLite 파일이 수개월에 걸쳐 불필요하게 팽창합니다.

작동 원리: POST /api/system/optimize-db 로 모든 DB 에 PRAGMA optimize 와 VACUUM 을 실행하고 확보 공간을 계산합니다.

이점: 쿼리 시간을 항상 밀리초 범위로 유지합니다.

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

66. 원자적 사전 쓰기 스냅샷 및 1-클릭 롤백(backend/snapshot_manager.py)

문제: 에이전트가 파일을 잘못 덮어쓰면 이전 상태 복원이 번거로웠습니다.

작동 원리: 쓰기 액세스 전 원본을 .hermes/snapshots/에 자동 백업하고 클릭·API 로 즉시 복원합니다. GitHub 풀·컨테이너 업데이트 전 모든 SQLite DB 의 사전 업데이트 스냅샷을 data/snapshots/에 추가로 통합합니다.

이점: *모든 파일 및 코드 변경에 대한 완전한 안전망.*

비용과 그림자(트레이드오프): 각 사전 쓰기 스냅샷은 파일 덮어쓰기 전 약 5–15ms 의 디스크 I/O 가 소모됩니다.

언제 비활성화합니까? 및 임곗값: 링 버퍼는 기본적으로 50 개 스냅샷으로 제한됩니다.

67. Git 호환 라이브 diff 미리보기(backend/diff_engine.py)

문제: 승인 전 전체 파일을 다시 읽는 대신 정확히 어떤 행이 변경되는지 보고 싶습니다.

작동 원리: 기존분과 에이전트 초안 간의 색상 행 비교(빨강 = 삭제, 초록 = 신규)를 생성합니다.

이점: *HITL 게이트에서 코드·문서 변경 사항을 초 단위로 검사합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

8. 보안, 제로 트러스트 거버넌스 및 시스템 진단(Pillar 6)

68. 암호화 체인 SHA-256 감사 로그 (backend/audit_log.py)

문제: 단순 텍스트 로그는 사후에 변경하거나 삭제할 수 있습니다.

작동 원리: 각 이벤트는 이전 항목의 SHA-256 해시와 연결(블록체인 원칙)되며 GET /api/audit/verify 로 변조 검사 가능합니다.

이점: *모든 시스템 및 승인 조치에 대한 감사 대응 증빙.*

비용과 그림자(트레이드오프): 최소 쓰기 오버헤드(항목당 약 2ms), 암호화 체인은 수개월에 걸쳐 성장.

언제 비활성화합니까? 및 임곗값: 감사 대응 거버넌스를 위해 항상 켜 둡니다.

69. Local Security Guard — 25 개 프롬프트 인젝션 및 탈옥 시그니처

문제: 악성 이메일이나 웹페이지에는 숨겨진 명령('Ignore previous instructions...')이 포함될 수 있습니다.

작동 원리: 외부 텍스트 입력을 에이전트 컨텍스트에 들어가기 전 25 개 공격 패턴으로 스캔합니다.

이점: *시스템 경계에서 간접 프롬프트 인젝션을 차단합니다.*

비용과 그림자(트레이드오프): 매우 엄격한 휴리스틱은 IT 보안 텍스트나 펜테스트 프롬프트에서 오탐(False Positive)을 유발할 수 있습니다.

언제 비활성화합니까? 및 임곗값: 승인된 레드팀 테스트용으로만 일시적으로 완화합니다.

70. 시크릿 마스킹 및 레드팀 출력 가드 (backend/secret_masker.py)

문제: API 키나 비밀번호가 로그, 보고서 또는 채팅 출력에 우발적으로 노출될 수 있습니다.

작동 원리: 발신 텍스트를 실시간 키 패턴(sk-..., Bearer, 비밀번호)으로 필터링해 [GESCHÜTZTER_KEY]로 교체합니다.

이점: *기밀 액세스 데이터의 우발적 노출을 방지합니다.*

비용과 그림자(트레이드오프): 공격적 정규식 마스킹은 드물게 코드 출력의 무해한 Base64·해시 문자열을 가릴 수 있습니다.

언제 비활성화합니까? 및 임곗값: 프로덕션 운영에서는 API 키 보호를 위해 항상 활성 상태로 유지.

71. 보안 Docker 소켓 프록시(내부 포트 2375)

문제: /var/run/docker.sock 직접 액세스는 컨테이너에 호스트 전체 무제한 루트 권한을 부여합니다.

작동 원리: 제한적 프록시 컨테이너를 중간에 두어 명시 허용된 상태·재시작 명령만 통과시킵니다.

이점: *제로 트러스트 원칙에 따라 호스트 OS 를 보호합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

72. A/B 모델 벤치마커 및 경험적 성능 스코어러(backend/model_benchmarker.py)

문제: 새 언어 모델이 매주 출시되며, 자체 업무 적합성은 종종 불분명합니다.

작동 원리: 두 모델을 동일 작업으로 병렬 테스트하고 응답 시간, 토큰 비용, 결과 품질을 비교합니다.

이점: *에이전트 플릿의 모델 선택을 위한 객관적 데이터 기반.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

73. 시스템 전체 360° 상태 점수 및 진단 엔진(backend/system_diagnostics.py)

문제: 개별 지표만으로는 전체 시스템 상태를 한눈에 파악하기 어렵습니다.

작동 원리: Docker 플릿 상태(35%), 동시성 큐(20%), 감사 무결성(20%), 교착 없음(15%), RAG 도달 가능성(10%)으로 0–100% 종합 점수와 실행 권고를 계산합니다.

이점: *GET /api/system/health-score 를 통한 운영체제 즉시 전체 진단.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

9. 자기 진화, 마켓플레이스 및 자동 문서화(Pillar 7)

74. 인사 기록 피드백이 있는 자율 스킬 통합(backend/skill_synthesizer.py)

문제: 성공적인 다단계 해결 경로는 지금까지 수동으로 새 스킬로 프로그래밍해야 했습니다.

작동 원리: 반복 성공 도구 체인을 감지해 SKILL.md 초안을 자동 생성하고 에이전트 인사 파일에 등록해 승인용으로 제출합니다.

이점: *시스템은 운영 중에 자체적으로 기능을 확장합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

75. 동적 인재·스킬 마켓플레이스 (skill_markt.js 및 AgentSkillsHub)

문제: 새 사업 분야용 역할이나 워크플로를 처음부터 새로 작성하는 것은 시간이 많이 소모됩니다.

작동 원리: 원클릭 가져오기·내보내기용 검색 가능한 카탈로그: 전체 에이전트 프로필과 외부 Skill(agentskillshub.dev 통합 포함).

이점: *새 프로젝트 시 에이전트 플릿을 수 초 만에 확장합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

76. 완전한 프론트엔드-백엔드 동기화(238 개 fetch / 581 개 경로 / 29 개 모달)

문제: 대규모 웹 앱에서 고아 버튼이나 연결되지 않은 모달은 종종 조용한 오류를 유발합니다.

작동 원리: 자동 검사 파이프라인이 238 개 프론트엔드 API 호출·29 개 모달이 368 개 백엔드 경로와 정확히 연결됨을 보장합니다.

이점: *모든 화면에서 0 개의 끊긴 링크와 100% 작동 컨트롤을 보장합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

77. 자율 마스터 서류철 및 매뉴얼 생성기(backend/system_dossier_generator.py)

문제: 아키텍처 문서는 작성 후 며칠 만에 구식이 되는 경우가 많습니다.

작동 원리: 버튼 클릭(POST /api/system/generate-dossier)으로 프로젝트 폴더와 Obsidian-Vault(00_DOKUMENTATION/)에 시스템 아키텍처 문서를 동시 생성합니다.

이점: *수동 관리 없이 시스템 문서를 항상 최신 상태로 유지합니다.*

비용과 그림자(트레이드오프): 대시보드에 영구 핵심 화면 또는 시스템 인프라로 통합됨(무시할 수준의 RAM/CPU 기본 사용량, 모듈 내 LLM 실행 시에만 토큰 비용 발생).

10. 참조 매트릭스: 제어 가능한 Evey/Hermes 플러그인

번호 및 플러그인 ID	전문 카테고리	운영체제에서의 핵심 메커니즘 및 기능
78. evey-cost-guard	보안 및 가드	일별·작업별 실시간 비용 모니터링과 강제 예산 차단.
79. evey-session-guard	보안 및 가드	세션 실행 시간 및 토큰 상한을 모니터링하고 좀비 세션을 방지합니다.
80. evey-email-guard	보안 및 가드	수신 이메일을 분류 전에 피싱·프롬프트 인젝션 대상으로 스캔합니다.
81. evey-watchdog	보안 및 가드	30 초 주기로 활성 프로세스를 점검하고 차단된 워커를 자동 재시작합니다.

82. evey-cache	성능 및 캐시	반복 쿼리의 즉시 무료 응답을 위한 24 시간 응답 캐시.
83. disk-cleanup	성능 및 유지보수	임시 아티팩트와 오래된 로그를 정리하고 디스크 공간을 확보합니다.
84. evey-telemetry	모니터링 및 분석	Mission Control 및 Guardian 용 토큰 흐름, 지연 및 모델 호출을 수집합니다.
85. evey-status	모니터링 및 분석	11 개 Docker 컨테이너와 게이트웨이에 대한 지속적인 상태 확인.
86. evey-delegation-score	모니터링 및 분석	에이전트 간 작업 인계의 성공률과 품질률을 평가합니다.
87. evey-autonomy	에이전트 인텔리전스	다단계 백그라운드 결정에서 자율성 수준을 제어합니다.
88. evey-council	에이전트 인텔리전스	여러 스페셜리스트 간의 다수결 및 합의 형성(마스터마인드 결합).
89. evey-reflect	에이전트 인텔리전스	결과 제출 전 자체 오류 수정을 위한 자기 성찰 루프.
90. evey-learner	에이전트 인텔리전스	learnings 테이블용으로 완료된 작업에서 지속적인 인사이트를 추출합니다.
91. evey-identity	에이전트 인텔리전스	SOUL.md, 어조, 역할 가드레일의 엄격한 준수를 보장합니다.
92. evey-habits	에이전트 인텔리전스	매일 08:00 스탠드업 브리핑 같은 반복 루틴을 관리합니다.
93. evey-proactive	에이전트 인텔리전스	에이전트가 감지된 병목에 대해 스스로 행동 제안을 할 수 있습니다.
94. evey-goals	계획 및 작업	칸반의 운영 개별 작업을 상위 기업 목표와 연결합니다.
95. evey-scheduler	계획 및 작업	시간 및 우선순위 기반 백그라운드 작업 스케줄링.
96. evey-bridge	브리지 및 인터페이스	FastAPI 모놀리스, Hermes 데몬 및 사이드카 간의 내부 데이터 허브.
97. evey-mqtt	브리지 및 인터페이스	IoT 및 실시간 상태 신호용 경량 메시지 프로토콜.
98. evey-moltbook	브리지 및 인터페이스	구조화된 워크북과 통합 노트를 동기화합니다.

© Michael Keller

99. evey-telegram-ux	브리지 및 인터페이스	Telegram 에서 호출명 라우팅(/robert, /anwalt) 및 대화형 승인 버튼을 제어합니다.
100. evey-github	브리지 및 인터페이스	저장소 작업, 커밋 분석, 이슈 추적을 위한 연동.
101. evey-research	도구 및 리서치	Research Lab 의 웹 스크래핑·출처 추출용 심층 탐색 드라이버.
102. evey-news	도구 및 리서치	업계 피드 및 시장 뉴스용 백그라운드 스캐너(Oracle News Radar).
103. evey-digest	도구 및 리서치	방대한 문서 모음을 간결한 경영 요약으로 압축합니다.
104. evey-sandbox	도구 및 실행	Showroom 내 생성된 Python 및 분석 코드용 격리 실행 환경.
105. evey-rag	기억 및 지식	AnythingLLM 및 로컬 지식 아카이브 연결용 벡터 검색 드라이버.
106. evey-memory-adaptive	기억 및 지식	컨텍스트 압박에 따른 단기/장기 기억의 동적 가중치.
107. evey-memory-consolidate	기억 및 지식	관련 기억 항목의 통합 및 중복 제거를 위한 야간 실행.
108. evey-wallet	재무 및 예산	API 잔액, 할당량, 비용 관련 트랜잭션 한도를 관리합니다.
109. evey-validate	품질 및 실행	출력의 스키마 준수, 필수 마커(SYNTHESE-BEREIT) 및 형식 규정을 검증합니다.
110. evey-verification	품질 및 실행	최종 승인 전 핵심 주장에 대한 팩트체크와 교차 검증을 수행합니다.
111. evey-delegate-model	품질 및 실행	하위 작업에 비용 및 성능 최적의 목표 모델을 자동 선택합니다.

11. 엔터프라이즈 및 복원력 아키텍처

78. 리플레이 및 타임 트래블 디버깅(backend/time_travel_debugger.py)

문제: 많은 도구 호출의 복잡한 에이전트 워크플로에서 사후 오류 진단은 텍스트 로그에서 보기 어렵고 실행 분기 전환이 불가능합니다.

작동 원리: 'execution_traces'에 입출력, 스냅샷, 토큰 소비와 함께 각 부분 단계를 저장합니다. 인터랙티브 리플레이(GET /api/traces/{task_id}), N 단계 분기 State-Forking(POST /api/traces/{task_id}/fork), 수정 파일 원클릭

롤백을 지원합니다.

이점: 전체 파이프라인 재시작 없이 대체 프롬프트·모델로 빠른 원인 분석과 무위험 What-if 실험.

비용과 그림자(트레이드오프): 파일 편집당 디스크 I/O 약 5–15ms 증가, 스냅샷 한도별 하드에 약 50–250MB 추가 공간 사용.

언제 비활성화합니까? 및 임곗값: 디스크가 극히 부족한 시스템이나 대량 파일 생성(예: 빌드 아티팩트) 시 끄십시오. | 임계값: 최대 스냅샷: < 10 = 짧은 기록 | > 200 = 높은 디스크 소비 (기본값: 50). 최대 추적 단계: 기본값 100.

79. 다중 모델 평가 매트릭스 및 A/B 벤치마커 (backend/model_benchmarker.py)

문제: 에이전트에 대한 모델 할당은 경험적 비교값 부족으로 주관적 가정에 기반하는 경우가 많습니다.

작동 원리: 동일 프롬프트를 구성된 2 개 LLM 에 동시 전송(POST /api/router/benchmark)해 지연, 토큰 소비, 응답 길이를 측정하고 기록을 'model_benchmarks'에 영속 저장합니다(GET /api/router/benchmark/history).

이점: 에이전트 역할별 객관적 데이터 기반 모델 선택(예: 속도/비용용 DeepSeek V3 vs. 복잡 아키텍처용 Claude 3.7).

비용과 그림자(트레이드오프): N 개 후보 모델은 테스트 실행당 N 배의 API 토큰이 소모됩니다.

언제 비활성화합니까? 및 임곗값: 언제 끕니까? 지속 운영이 아닌 모델 비교용으로만 선택적으로 시작.

80. Tri-Agent 합의 게이트 및 중재 재판소 (backend/consensus_gate.py)

문제: 중요 개입(코드 리팩터링, 마이그레이션)은 단독 결정 시 환각·보안 공백 위험이 있습니다.

작동 원리: 3 인 위원회(시스템 아키텍트, 보안 책임자, QA 리드)를 소집합니다 (POST /api/consensus/council-vote). 각 리뷰어는 독립 투표하며 승인은 2/3 다수 필요. 모든 결정은 'consensus_decisions'에 보관됩니다(GET /api/consensus/history).

이점: 민감한 시스템 변경에서 Single-Point-of-Failure 환각의 안정적 위험 최소화·제거.

비용과 그림자(트레이드오프): 토큰 소비를 3 배로 늘리고 응답 시간을 +6~20 초 증가시킵니다. 만장일치 강제 시 결정 마비(Deadlock) 위험.

언제 비활성화합니까? 및 임곗값: 속도·낮은 API 비용 우선 시 끄십시오(또는 우선순위 4 = Critical 부터 활성화). | 임계값: 기준 우선순위: Prio 2 = 거의 모든 작업 3 배 토큰 | Prio 4 = 중요 작업만 (기본값: Prio 3). 다수결: 2/3 다수결 권장.

81. Git 롤백을 적용한 자율 작업 자가 치유(backend/task_self_healer.py)

문제: 구문 오류나 잘못된 코드 임포트는 작업을 즉시 차단하고 사용자 수동 개입이 필요합니다.

작동 원리: 스택트레이스를 격리하고 실행 오류 시 수정 파일의 Git 롤백을 수행한 뒤 구조화된 수정 프롬프트를 에이전트에 전달합니다(POST /api/healing/attempt). 최대 3 회 자율 수리 시도, 로그 조회는 GET /api/healing/logs.

이점: 코드·스크립트 실행의 최대 복원력, HITL 보드 에스컬레이션 전 자동 자가 복구.

비용과 그림자(트레이드오프): 자가 치유 시도마다 1 회 추가 LLM 호출(+3~12 초 지연·토큰 비용). 자동 Git 롤백은 실패 시 유용한 부분 성과도 버립니다.

언제 비활성화합니까? 및 임곗값: 미완성 코드 초안을 수동 후속 작업용으로 작업 디렉터리에 남겨야 하면 Git 롤백을 끄십시오. | 임계값: 최대 수리 시도: 1 = 즉시 포기 | > 5 = 고비용 수리 캐스케이드 (기본값: 3).

82. 이벤트 메시 및 받은편지함 디렉터리 워처 (backend/event_mesh.py)

문제: 지금까지 작업은 대시보드 수동 시작이나定时 IMAP 폴링 조회로 시작해야 했습니다.

작동 원리: 중앙 인바운드 이벤트 버스: 외부 웹훅(POST /api/events/inbound/{channel})과 자동 디렉터리 워처(POST /api/events/scan-directory)가 'data/inbox/'의 새 파일을 Kanban triage 작업으로 직접 전환합니다.

이점: 외부 시스템(GitHub, n8n, Stripe, 로컬 파일 스캐너)의 완전 자동 이벤트·증빙 수집(사용자 개입 없음).

비용과 그림자(트레이드오프): 매우 짧은 스캔 주기(< 3 초)는 지속 파일 시스템 폴링을 일으키고, 일일 아침 스탠드업은 실행당 약 1,500–3,000 토큰을 소비합니다.

언제 비활성화합니까? 및 임곗값: 로컬 수신함 폴더를 적극 사용하지 않거나 아침 브리핑 보고가 필요 없으면 끄십시오. | 임계값: 스캔 주기: 1–2 초 = 불필요 디스크 폴 | > 30 초 = 파일 감지 지연 (기본값: 5 초). 스탠드업 시간: 기본값 08:00.

12. 자율성, 샌드박스 및 거버넌스 아키텍처

83. 임시 코드 샌드박스(backend/ephemeral_sandbox.py)

문제: 에이전트 코드 실행은 호스트의 시스템 충돌, 무한 루프, 의도치 않은 파일 변경 위험을 수반합니다.

작동 원리: 엄격한 자원 상한(512MB RAM, 1 CPU, 30 초 타임아웃, 네트워크 없음)으로 단명 격리 Docker 마이크로 컨테이너에서 코드 실행(POST /api/sandbox/execute). 샌드박스는 완료 후 흔적 없이 삭제됩니다.

이점: 자동화된 코드 실행 및 테스트에서의 절대적 보안과 시스템 안정성.

비용과 그림자(트레이드오프): 컨테이너 콜드 스타트는 도구 호출당 약 300–800ms 추가 지연. 네트워크 비활성 시 pip install·외부 API 호출이 스크립트에서 즉시 실패합니다.

언제 비활성화합니까? 및 임곗값: 신뢰할 수 있는 로컬 스크립트가 직접 호스트 액세스·빠른 패키지 설치가 필요하면 끄십시오(또는 네트워크 활성화). | 임계값: RAM 한도: < 256MB = Pandas/NumPy OOM-Kill | > 2048MB = 호스트 RAM 리스크 (기본값: 512MB). 타임아웃: < 10 초 시 빌드 중단(기본값: 30 초).

84. 야간 기억 통합 (backend/memory_consolidator.py)

문제: 계속 증가하는 추적, 하트비트, 오래된 대화 데이터는 장기 운영에서 DB 쿼리를 느리게 합니다.

작동 원리: 휴면 정리(기본 03:00): 보관 기한(settings.json) 기준 오래된 추적·비중요 로그를 정리합니다. Hermes Core 설정(시간, 보관 일수, 가지치기 깊이, 수동 실행 버튼)으로 완전 제어됩니다.

이점: 영구적으로 가벼운 DB, 짧은 지연, 수동 유지보수 없는 자동 시스템 위생.

비용과 그림자(트레이드오프): 통합 실행 중(~10–30 초) 짧은 CPU/디스크 부하. 가지치기 임계값이 너무 낮으면(< 0.88) 관련 있지만 독립적인 팩트를 합칠 수 있습니다.

언제 비활성화합니까? 및 임곗값: 법의학 감사용 원시 추적을 영구 무필터 보존해야 하면 끄십시오. | 임계값: 보관: < 7 일 = 중요 감사 흔적 손실 | > 60 일 = 대형 SQLite DB (기본값: 14 일). 프루닝 임곗값: 0.95 = 보수적 | 0.88 = 공격적(기본값: 0.92).

85. 스크린샷 감사를 포함한 비주얼 웹 에이전트 (backend/visual_web_agent.py)

문제: API 없는 복잡한 웹 포털(SPA, 양식, 인터랙티브 대시보드)의 작업은 텍스트 스크레이퍼로 처리할 수 없습니다.

작동 원리: 실제 헤드리스 Chromium 브라우저 조종(POST /api/browser/browse-and-capture), .hermes/visual_traces/{task_id}/에 클릭·입력 단계별 스크린샷 캡처, 구조 데이터 추출.

이점: API 액세스 없는 최신 웹 애플리케이션과의 빈틈없는 시각적 추적 및 자동 상호작용.

비용과 그림자(트레이드오프): Headless-Chromium 은 활성 감사 실행당 약 300MB RAM 과 약 3–5 초 렌더링 시간이 필요합니다.

언제 비활성화합니까? 및 임곗값: 언제 끕니까? UI 검증 없는 순수 백엔드 운영 시.

86. 가상 워룸 및 프리모템 스트레스 테스트(backend/war_room_simulator.py)

문제: 예상 못한 보안·수용·컴플라이언스 리스크는 제품 출시 후에야 뒤늦게 발견되는 경우가 많습니다.

작동 원리: 승인 전 3 개 적대 당사자(비판적 대고객, 공격적 펜테스터, 엄격한 컴플라이언스 감사관)를 시뮬레이션합니다(POST /api/simulation/war-room). 구속력 있는 조건의 집계 리스크 매트릭스(LOW~CRITICAL)를 작성합니다.

이점: 라이브 전환 전 취약점의 사전 발견 및 비용이 큰 오판 방지.

비용과 그림자(트레이드오프): 약 1,500–3,000 추가 토큰과 실제 작업 전 +4~10 초 대기. 차단 임계값 MEDIUM 에서 오경보가 잦아집니다.

언제 비활성화합니까? 및 임곗값: 일상 개발에서 Auto-Pre-Mortem 을 끄고 프로덕션 중요 개입(HIGH/CRITICAL)에만 사용하십시오. | 임계값: 차단 임계값: MEDIUM = 빈번 사전 차단 | CRITICAL = 치명 리스크만 중단 (기본값: HIGH).

87. 에이전트 간 SLA 및 인계 계약(backend/sla_contract_enforcer.py)

문제: 다중 에이전트 계층의 느슨한 위임은 불완전한 납품으로 이어져 인간 오퍼레이터에게 에스컬레이션되는 경우가 많습니다.

작동 원리: 내부 위임 시 의무 필드, 최소 길이, 수락 기준의 디지털 계약을 체결합니다(POST /api/contracts/create). 납품을 자율 검증하고(POST /api/contracts/validate) 미흡 결과를 자동 재작업으로 반려합니다.

이점: 에이전트 팀의 품질 표준 엄격 준수와 HITL 보드의 미완성 중간 단계 방지.

비용과 그림자(트레이드오프): 최소 길이를 너무 높게 설정하면(> 300 자) 정확한 단답이 거부되고 고비용 재작업 루프가 발생합니다.

언제 비활성화합니까? 및 임곗값: 서브에이전트가 간결한 상태·예/아니오 답변을 반환해도 되면 Auto-Reject 를 끄십시오. | 임계값: 최소 길이: < 30 = 플레이스홀더 통과 | > 300 = 간결 팩트 답변 거부 (기본값: 100 자). 최대 처리 시간: 기본값 60 분.

88. 중앙 자율성 및 임곗값 제어(backend/autonomy_settings.py)

문제: 프로그램 코드의 고정 임계값은 운영 중 보안·샌드박스·복원력 규칙의 빠른 조정을 막습니다.

작동 원리: 10 개 자율 그룹(동시성, 루프 킬러, 자가 치유, Docker 샌드박스, 타임 트래블 스냅샷, 합의 게이트, War-Room, SLA 인계 계약, 수신함/스탠드업, 플릿 위생)을 settings.json 에 묶고 GET/POST

/api/settings/autonomy-modules 로 System Control Center 와 동기화합니다.

이점: 모든 보호·자율 모듈을 실행 중 클릭으로 켜고 끄며 원하는 위험·자원 프로필에 정밀 조정할 수 있습니다.

비용과 그림자(트레이드오프): 결함 컨테이너는 자동 재시작으로 크래시 루프에 빠질 수 있으며, 시각 브라우저 감사는 Chromium 을 시작합니다(+300MB RAM, +4 초 소요).

언제 비활성화합니까? 및 임곗값: 수동 컨테이너 유지보수 중 Auto-Restart 를 끄고, 순수 백엔드 운영 시 브라우저 감사를 비활성화하십시오. | 임계값: DB 백업 일수: < 3 일 = 짧은 기록 | > 30 일 = 약 500MB+ 백업 저장소 (기본값: 7 일).

13. 성능 코어, 처리량 확장 및 토큰 효율성

89. 시맨틱 프롬프트 및 응답 캐싱 (backend/semantic_cache.py)

문제: 반복·약간 변형된 에이전트 요청은 불필요한 LLM API 비용과 수초 대기를 유발합니다.

작동 원리: 각 프롬프트의 SHA-256 해시와 L2 정규화 단어/3 그램 벡터를 SQLite 에 계산하고 코사인 유사도 >= 0.95 시 저장된 응답 적중을 즉시 제공합니다.

이점: 시맨틱 동일 쿼리에서 10ms 미만 응답 시간과 100% 토큰 절감.

비용과 그림자(트레이드오프): 코사인 임계값이 너무 낮으면(< 0.92) 캐시가 미세 의미 차이(예: '테스트 DB 삭제' vs. '프로덕션 DB 삭제')를 놓치고 잘못·오래된 답변을 제공합니다.

언제 비활성화합니까? 및 임곗값: 실시간 데이터(주가, 실시간 로그, 시스템 상태) 작업이나 모든 답변이 새로 생성되어야 하면 끄십시오. | 임계값: 코사인 임계값: < 0.92 = 캐시의 오답! | > 0.98 = 적중 희박 (기본값: 0.95). TTL: 기본값 24 시간. 최대 항목: 기본값 500.

90. 병렬 Map-Reduce 하위 작업 / Fan-Out 및 Fan-In(backend/map_reduce_executor.py)

문제: 많은 개별 문서·부분 질문의 순차 처리는 선형 시간 증가와 긴 대기 시간을 초래합니다.

작동 원리: 목록 주문을 자동 N 병렬 Asyncio 워커(세마포어 제어)로 나누고 Reduce 단계에서 부분 결과를 합칩니다.

이점: 이벤트 루프 과부하 없이 문서·리서치·코드 일괄 분석 처리량을 배가합니다.

비용과 그림자(트레이드오프): N 배 동시 API 부하(HTTP-429 속도 제한 위험)와 후속 Reduce 종합의 약 20–35% 토큰 오버헤드를 발생시킵니다.

언제 비활성화합니까? 및 임곗값: 엄격한 API 속도 제한이나 하위 작업이 밀접 의존해 순차 수행되어야 하면 끄십시오. | 임계값: 최대 워커: > 8 = 속도 제한 위험 (기본값: 4). Chunk-Timeout: 기본값 30 초. 항목 수 기준: 기본값 3.

91. 추측성 고속 라우팅 및 지연 헤징 (backend/speculative_hedger.py)

문제: 개별 느린 클라우드 요청(테일 지연 스파이크)이 전체 에이전트 실행을 차단합니다.

작동 원리: hedge_delay_seconds(예: 2.5 초) 경과 후 빠른 폴백 모델에 병렬 대체 호출을 자동 시작하고 먼저

도착한 결과를 인수합니다.

이점: 느린 API 엔드포인트로 인한 정지를 제거하고 99 번째 백분위수 응답 시간을 안정화합니다.

비용과 그림자(트레이드오프): 헤지 시작 직후 주 모델이 응답하면 두 요청 모두 과금됩니다(느린 프롬프트 시 최대 2 배 토큰 비용).

언제 비활성화합니까? 및 임곗값: 부족한 API 예산이나 정기적으로 10 초 이상 사고 시간이 필요한 느린 추론 모델 사용 시 끄십시오. | 임계값: 헤지 지연: < 1.5 초 = 거의 항상 이중 API 비용! | > 8 초 = 지연 보호 희박 (기본값: 2.5 초).

92. 동적 컨텍스트 압축 및 계층적 요약(backend/context_compactor.py)

문제: 긴 에이전트 세션은 컨텍스트 창을 부풀리고 프롬프트 프리필을 늦추며 턴당 비용을 급증시킵니다.

작동 원리: 설정 가능한 토큰 임계값부터 모든 하드 팩트(목표, 파일 경로, 결정, 오류)를 간결한 [HIERARCHICAL_MEMORY_STATE] 블록에 추출하고 최근 라운드만 전문 유지합니다.

이점: 중요 프로젝트 데이터 손실 없이 긴 세션에서 컨텍스트 토큰 65–80% 절감.

비용과 그림자(트레이드오프): 압축 실행은 1 회 약 1,000 토큰. 오래 전 메시지의 정확한 코드 줄·작은 세부값은 요약에서 손실될 수 있습니다.

언제 비활성화합니까? 및 임곗값: 에이전트가 동일 세션의 아주 초기 메시지를 그대로 참조해야 하면 끄십시오. | 임계값: 토큰 임계값: < 3000 = 세부 조기 망각 | > 24000 = 높은 프롬프트 비용 (기본값: 6000). 최근 턴 유지: 기본값 4.

93. SQLite WAL 연결 풀 및 비동기 쓰기 배칭(backend/async_write_batcher.py)

문제: 병렬 에이전트 실행의 수백 개 개별 SQLite INSERT 는 디스크 I/O 스파이크와 'database is locked' 충돌을 일으킵니다.

작동 원리: 텔레메트리·감사 이벤트를 스레드 안전 RAM 링버퍼에 모아 executemany()로 WAL 모드 일괄 기록합니다.

이점: 디스크 쓰기 액세스를 약 80% 줄이고 메인 이벤트 루프를 블로킹 없이 유지합니다.

비용과 그림자(트레이드오프): 다음 플러시 주기 전 하드 정전·kill -9 시 RAM 버퍼의 최근 1–5 초 항목이 손실됩니다.

언제 비활성화합니까? 및 임곗값: 모든 개별 로그 항목이 초 지연 없이 디스크에 즉시 동기 기록되어야 하면 끄십시오. | 임계값: 배치 크기: < 5 = 많은 디스크 쓰기 | > 100 = 충돌 시 손실 증가 (기본값: 25). 플러시 간격: 기본값 2.0 초.

14. 현대적 메모리 및 RAG 아키텍처

94. BM25 FTS5 및 Reciprocal Rank Fusion 하이브리드 검색(backend/hybrid_search_engine.py)

문제: 순수 벡터 검색은 의미 유사 텍스트를 찾지만 정확한 오류 코드·함수명·일련번호에는 자주 실패합니다.

작동 원리: SQLite FTS5 전체 텍스트 랭킹(BM25)과 코사인 벡터 유사도를 RRF(Reciprocal Rank Fusion)로

결합합니다.

이점: 정확한 키워드 일치와 시맨틱 이해를 하나의 고정밀 결과 목록으로 통합합니다.

비용과 그림자(트레이드오프): 이중 인덱스 조회는 검색 지연을 약 15–40ms 높이고, Top-K 가 너무 높으면(> 10) 에이전트 프롬프트가 수천 추가 토큰으로 찹니다.

언제 비활성화합니까? 및 임곗값: 거의 끄지 마십시오 — 토큰당 비용 최소화 시 Top-K 를 3–5 로 낮추는 정도. | 임계값: RRF 상수 K: 기본값 60. BM25 가중치: 0.0 = 벡터만 | 0.5 = 균형 | 1.0 = 키워드만 (기본값: 0.50). Top-K: 기본값 5.

95. 로컬 지식 그래프 / 트리플 저장소(backend/knowledge_graph_store.py)

문제: 평탄한 텍스트 청크는 구조적 의존성(누가 어떤 시스템을 관리하는지, 어떤 구성 요소가 어떤 서비스를 쓰는지)을 포착하지 못합니다.

작동 원리: 주어-술어-목적어 트리플을 신뢰도 점수와 함께 SQLite 에 추출·저장하고 다단계 그래프 탐색(Multi-Hop)을 지원합니다.

이점: 에이전트는 여러 계층에 걸친 시스템 및 조직 연관성을 인식합니다.

비용과 그림자(트레이드오프): 자동 트리플 추출은 저장 시 약 500 추가 토큰이 듭니다. 2 홉 초과 탐색부터 이웃 노드 수가 기하급수적으로 폭발합니다.

언제 비활성화합니까? 및 임곗값: 순수 단기 채팅의 장기 지식 구축 없으면 자동 추출을 끄고 홉을 최대 2 로 제한하십시오. | 임계값: 최대 홉: 1 = 직접 이웃 | 2 = 최적 | >= 3 = 노드 폭발! (기본값: 2). 최소 신뢰도: 기본값 0.70.

96. Agentic Corrective RAG / CRAG (backend/corrective_rag.py)

문제: 기존 RAG 는 무관한 검색 결과를 여과 없이 언어 모델에 전달해 오답을 유발합니다.

작동 원리: 검색된 각 문서 청크를 관련성 임계값(0.45)으로 평가하고 약한 적중 시 동의어 확장 쿼리 재작성으로 자동 수행합니다.

이점: 답변 생성 전 지식 노이즈를 걸러내고 불명확한 질문의 적중 품질을 높입니다.

비용과 그림자(트레이드오프): 쿼리 재작성마다 1 회 추가 LLM 호출(+1.5~4 초 지연). 관련성 임계값이 너무 높으면(> 0.70) CRAG 가 불필요한 추가 탐색을 돕니다.

언제 비활성화합니까? 및 임곗값: 지식 DB 가 작고 정돈됐거나 2 초 이내 빠른 응답이 필요하면 끄십시오. | 임계값: 관련성 임계값: < 0.30 = 약한 출처 통과 | > 0.70 = 무한 검색 루프 (기본값: 0.45). 최대 재작성: 기본값 2.

97. MemGPT/Letta 방식 2 단계 코어 메모리(backend/core_memory_manager.py)

문제: 중요 사용자 선호와 프로젝트 목표는 단명한 채팅 창에서 사라지면 손실됩니다.

작동 원리: 편집 가능한 고정 작업 기억 블록(<CORE_WORKING_MEMORY>)을 대형 아카이브와 분리해 시스템 프롬프트에 영구 고정합니다.

이점: 임의로 많은 세션에 걸쳐 에이전트 스스로 관리하는 영구 핵심 기억을 보장합니다.

비용과 그림자(트레이드오프): 고정 블록마다 모든 LLM 호출에서 영구 프롬프트 토큰을 점유합니다(10 블록 × 800 자 ≈ 메시지당 약 2,000 토큰 기본 부하).

언제 비활성화합니까? 및 임곗값: 작은 컨텍스트 모델·부족한 토큰 예산 사용 시 차단 수를 3–5 로 줄이거나 끄십시오. | 임계값: 최대 블록: > 15 = 높은 토큰 기본 소비 (기본값: 10). 블록당 최대 문자: < 300 = 규칙 절단 | > 1500 = 프롬프트 막힘 (기본값: 800).

98. 자동 인용 및 출처 고정 (backend/citation_pinner.py)

문제: 직접 출처 참조가 없으면 에이전트의 팩트 주장이 어떤 문서에서 왔는지 사용자가 검증할 수 없습니다.

작동 원리: 토큰 중첩으로 답변 문장을 검색 출처 문서에 매핑하고 인라인 근거([Quelle: ID])를 삽입해 최소 근거 비율을 검사합니다.

이점: 모든 진술을 감사 대응 추적 가능하게 하고 근거 없는 주장을 즉시 드러냅니다.

비용과 그림자(트레이드오프): 출력 토큰 소비를 약 10–15% 늘립니다. 최소 커버리지가 너무 높으면(> 80%) 창의적·요약 질문에 오경보가 발생합니다.

언제 비활성화합니까? 및 임곗값: 자유 브레인스토밍, 코드 생성 또는 문서와 무관한 일상 채팅 시 비활성화. | 임계값: 최소 커버리지: < 30% = 근거 없는 주장 | > 85% = 요약 시 오경고 (기본값: 60%).

15. 신뢰할 수 있는 실행 및 결정론(듀러블 실행)

99. 구조화된 출력 및 JSON 스키마 보장(backend/structured_output_enforcer.py)

문제: 언어 모델은 가끔 유효하지 않은 JSON(Markdown 울타리, 누락 괄호, 잘못된 형식)을 제공해 후속 코드 파이프라인이 충돌합니다.

작동 원리: 원시 문자열의 구문 오류를 자동 수정하고 대상 스키마의 모든 필수 필드·데이터 형식을 강제합니다.

이점: 에이전트와 백엔드 인터페이스 간 100% 기계 판독 가능하고 타입 안전한 JSON 페이로드를 보장합니다.

비용과 그림자(트레이드오프): 자동 수리 루프마다 +1~3 초 지연과 추가 토큰. 엄격한 스키마 검사는 무해한 추가 필드로도 답변을 거부합니다.

언제 비활성화합니까? 및 임곗값: 엄격 검증을 끄면 API 엔드포인트·스크립트가 유연·부분 JSON 필드로도 동작할 수 있습니다. | 임계값: 스키마 검사: 엄격 vs. 관용 (기본값: 엄격). 최대 수리: 1–2 = 오타 98% 수정 | > 3 = 토큰 낭비 (기본값: 2).

100. 듀러블 실행 체크포인팅 / 일시 정지 및 재개(backend/durable_checkpoint_manager.py)

문제: 10 단계 리서치·빌드 주문이 재시작으로 8 단계에서 중단되면 지금까지 7 개 사전 작업 모두를 비싸게 반복해야 했습니다.

작동 원리: 완료된 각 단계 후 전체 변수·진행 상태를 SQLite 에 트랜잭션으로 보호합니다.

이점: 중단된 장기 실행은 유효한 마지막 체크포인트에서 정확히 재개되며 작업·토큰 이중 소비가 없습니다.

비용과 그림자(트레이드오프): 매 개별 단계 체크포인팅(N = 1)은 눈에 띄는 SQLite 쓰기 부하. 부팅 시 자동 재개가 전날 오래된 작업을 원치 않게 재점화할 수 있습니다.

언제 비활성화합니까? 및 임곗값: 중단된 작업이 재시작 후 수동 검토·승인되어야 하면 Auto-Resume 을

끄십시오. | 임계값: N 단계마다: 1 = 절대 충돌 안전, 디스크 I/O 증가 | 3–5 = SSD 보호 (기본값: 1). 보관: 기본값 14 일.

101. 멱등성 키 및 도구 호출 중복 제거(backend/idempotency_guard.py)

문제: 네트워크 재시도·에이전트 루프 시 동일 이메일·청구서·웹훅 동작이 중복 발생 위험이 있습니다.

작동 원리: 변경 도구 호출마다 SQLite 원장에 SHA-256 Idempotency 키를 부여하고 TTL 내 동일 후속 호출을 차단합니다.

이점: 현대 결제 게이트웨이(Stripe 원칙)를 본떠 중복된 외부 효과를 안정적으로 방지합니다.

비용과 그림자(트레이드오프): 사용자가 차단 기간(기본 24 시간) 내 동일 명령을 고의로 다시 실행하려 하면 가드가 차단합니다.

언제 비활성화합니까? 및 임곗값: 테스트 운영에서 동일 동작을 연속 반복 실행해야 하면 TTL 을 1 시간으로 낮추거나 일시 끄십시오. | 임계값: TTL(시간): < 1 시간 = 짧은 중복 보호 | > 48 시간 = 의도된 반복 차단 (기본값: 24 시간).

102. 에이전트 작업용 드라이런 / 섀도 모드(backend/dry_run_simulator.py)

문제: 새롭거나 복잡한 에이전트 주문은 실제 프로덕션 데이터에 날것으로 실행하기 전 위험 없이 테스트하고 싶어 합니다.

작동 원리: 섀도 모드에서 모든 쓰기 파일·DB·API 명령을 가로채 대신 상세 변경 미리보기 계획을 기록합니다.

이점: 최종 승인 전 안전한 드라이런 및 리스크 감사를 가능하게 합니다.

비용과 그림자(트레이드오프): 활성 상태에서는 실제 파일이 기록되지 않고 실제 API 가 실행되지 않습니다! 스위치를 잊으면 시스템이 겉보기에 '죽은' 듯 보입니다.

언제 비활성화합니까? 및 임곗값: 정상 운영에서는 반드시 꺼두십시오! 드라이런·보안 감사용으로만 표적 활성화하십시오. | 임계값: 메인 스위치 기본 꺼짐(false). 격리 테스트 실행용으로만 활성화.

103. 자동 리플레이 관리자가 있는 Dead-Letter-Queue(DLQ)(backend/dead_letter_queue.py)

문제: 실패한 웹훅·비동기 백그라운드 작업은 일시적 제 3 자 장애 시 수집함 없이 흔적 없이 손실됩니다.

작동 원리: 실패한 모든 작업을 원본 페이로드·실패 사유와 함께 DLQ 테이블에 저장하고 지수 백오프로 재시도합니다.

이점: 어떤 주문도 손실되지 않으며, 실패한 작업은 언제든 개별·일괄 재실행할 수 있습니다.

비용과 그림자(트레이드오프): 영구 결함 주문(예: 유효하지 않은 API 키)은 재시도로 자원을 불필요 소모하며, 지연 재실행이 작업을 뒤늦게 촉발할 수 있습니다.

언제 비활성화합니까? 및 임곗값: 실패한 주문을 수동으로만 검토·개별 승인해야 하면 Auto-Replay 를 끄십시오. | 임계값: 최대 재시도: 1 = 짧은 장애에 중단 | > 5 = 죽은 작업 적체 (기본값: 3). 백오프 기준: 기본값 10 초(10 초 -> 20 초 -> 40 초).

16. 보안, 데이터 보호 및 엔터프라이즈 가드레일

104. 양방향 PII 스크러버 및 재하이드레이션 (backend/pii_scrubber.py)

문제: 이메일·전화번호·IBAN 이 평문으로 외부 클라우드 LLM 에 전송되면 DSGVO 위반·의도치 않은 유출 위험이 있습니다.

작동 원리: API 호출 전 민감 개인정보를 로컬 플레이스홀더([EMAIL_1], [IBAN_1])로 교체하고 수신 답변에서 실제 값으로 다시 바꿉니다.

이점: 클라우드 모델은 실제 개인정보를 절대 보지 않지만 사용자는 현장에서 완전한 문서·답변을 받습니다.

비용과 그림자(트레이드오프): LLM 이 응답 텍스트의 플레이스홀더를 살짝 변형하면 역번역(rehydration)이 실패합니다. 정규식 마스킹이 코드 문자열을 변형할 수도 있습니다.

언제 비활성화합니까? 및 임곗값: 로컬 모델(Ollama)만 사용하거나 LLM 이 실제 이메일 도메인/이름을 반드시 분석해야 하면 끄십시오. | 임계값: 이메일, 전화번호, IBAN, 자동 재수화의 세분 스위치(모두 기본 활성).

105. 에이전트용 이그레스 방화벽 및 도메인 허용 목록(backend/egress_firewall.py)

문제: 웹페이지의 숨겨진 프롬프트 인젝션으로 에이전트가 내부 데이터를 HTTP 요청으로 외부 공격 서버에 보내도록 지시받을 수 있습니다.

작동 원리: 구성된 도메인 허용 목록에 대해 모든 발신 대상 URL 을 검사하고 미승인 호스트·원시 IPv4 를 차단합니다.

이점: 조작된 입력에도 네트워크 수준에서 데이터 유출을 차단합니다.

비용과 그림자(트레이드오프): allowlist_only 활성 시 도메인 목록에 명시되지 않은 새 웹·API·문서는 웹 조사에서 강제 차단됩니다.

언제 비활성화합니까? 및 임곗값: Research 에이전트가 개방 웹에서 자유 조사해야 하면 끄십시오(또는 필요 도메인 추가). | 임계값: 허용 도메인: 쉼표 구분 화이트리스트. 원시 IP 차단은 직접 IP 유출을 차단합니다.

106. 세분화된 에이전트 RBAC 및 파일 스코프 제일 (backend/agent_rbac_jail.py)

문제: 하위 에이전트가 전체 프로젝트 디렉터리에 완전 쓰기 권한이 있으면 프론트엔드 에이전트가 실수로 백엔드 커널 파일·.env 키를 덮어쓸 수 있습니다.

작동 원리: 각 에이전트 역할을 고정 도구 목록과 허용 디렉터리 제일(예: 프론트엔드용 static/)에 바인딩합니다.

이점: 운영체제의 각 프로세스에 최소 권한 원칙(Least Privilege)을 적용합니다.

비용과 그림자(트레이드오프): 관리자 에이전트가 프로젝트 폴더 밖 설정 파일을 합법적으로 관리하거나 .env 템플릿을 직접 검사하지 못하도록 방지합니다.

언제 비활성화합니까? 및 임곗값: 전용 시스템 유지보수 작업에만 일시 완화하십시오. .env 보호는 정상 운영에서 항상 활성 유지해야 합니다. | 임계값: Path-Jail, .env 잠금, 시스템 폴더 잠금(/etc, /root, /proc) 스위치.

107. 결정적 출력 린터 및 정책 검사기(backend/output_policy_linter.py)

문제: 언어 모델은 시스템 프롬프트에도 가끔 인위적 AI 상투어나 위험한 셸 스니펫을 생성합니다.

작동 원리: 정규식으로 생성 텍스트를 실시간 스캔해 아첨 오프너/AI-Slop 을 제거하고 파괴 명령을 무력화합니다.

이점: 추가 LLM 검증 호출 없이 언어적 정밀도와 기술적 안전성을 보장합니다.

비용과 그림자(트레이드오프): 파괴 필터는 DROP·rm -rf 같은 명령이 포함되면 무해한 정리 스크립트·의도된 DB 마이그레이션도 차단할 수 있습니다.

언제 비활성화합니까? 및 임곗값: 표적 서버 정리 스크립트·SQL 리셋 마이그레이션을 만들 때 Destruktiv 차단기를 잠시 끄십시오. | 임계값: AI 상투어 제거, 파괴 명령 차단, 자동 정리 스위치.

108. 변조 방지 실행 영수증 / SHA-256 영수증(backend/execution_receipt_ledger.py)

문제: 컴플라이언스 감사(EU AI Act / ISO 42001)에 단순 텍스트 로그는 사후 변경 가능해 불충분합니다.

작동 원리: 작업별 정산(모델, 토큰, USD 비용, 실행 도구)을 블록체인 유사 체인에서 이전 항목의 SHA-256 해시와 연결합니다.

이점: 비용·결정 기록의 사후 조작은 무결성 검사에서 즉시 드러납니다.

비용과 그림자(트레이드오프): 최소 연산·저장 오버헤드(작업당 약 2ms·약 2KB SQLite 저장), 해시 체인은 수개월 지속 증가합니다.

언제 비활성화합니까? 및 임곗값: 운영 환경에서 영구 활성화 가능하며, 메모리가 극히 제한된 초소형 시스템에서만 끄십시오. | 임곗값: 해시 체인(SHA-256) + 실행별 상세 비용 분석.

17. 자기 최적화, 평가 및 군집 지능

109. DSPy/TextGrad 원리 기반 자동 프롬프트 최적화기(backend/prompt_optimizer.py)

문제: 정적 시스템 프롬프트는 사람이 수동으로 수정할 때까지 같은 실수를 계속 반복합니다.

작동 원리: 역할의 실제 오류 로그를 평가해 체계적 약점을 감지하고 시스템 프롬프트용 맞춤형 가드레일 추가 규칙을 생성합니다.

이점: 에이전트는 실패한 실행에서 지속 학습하고 지침을 스스로 강화합니다.

비용과 그림자(트레이드오프): 자동 적용 활성 시 잘못 도출된 규칙이 에이전트 행동을 영구 왜곡(프롬프트 드리프트)하고 시스템 프롬프트를 부풀릴 수 있습니다.

언제 비활성화합니까? 및 임곗값: 운영 자동 적용은 끄고 제안된 프롬프트 패치를 인간 검토 후 인수하십시오. | 임계값: 오류 기점: 1 = 개별 오류에 과민 | > 8 = 학습 지체 (기본값: 3). 최대 규칙: > 10 시 시스템 프롬프트 팽창(기본값: 5).

110. 골드 스탠더드 회귀 테스트 스위트 / 평가 러너(backend/eval_regression_runner.py)

문제: 시스템 프롬프트·모델의 겉보기 작은 변경도 보안·형식 규칙을 눈치채지 못하게 악화시킬 수 있습니다.

작동 원리: 새 설정 인수 전 자동 Gold-Standard 테스트 케이스를 실행해 합격률·금지 패턴을 검사합니다.

이점: 객관적 품질 게이트로 프로덕션 운영의 품질 저하를 방지합니다.

비용과 그림자(트레이드오프): 전체 Eval 실행마다 저장된 모든 테스트 프롬프트를 LLM API 에

전송합니다(규모에 따라 약 5,000–25,000 토큰, 15–60 초 소요).

언제 비활성화합니까? 및 임곗값: 빠른 실험 프롬프트 테스트를 대기 없이 수행해야 하면 배포 차단을 끄십시오. | 임계값: 최소 합격률: < 75% = 악화 통과 | 100% = 미세 편차도 차단 (기본값: 90%).

111. Multi-Armed Bandit 모델 라우터 (backend/bandit_model_router.py)

문제: 고정 모델 할당은 클라우드 공급자가 갑자기 느려지거나 오류율이 오르면 반응하지 않습니다.

작동 원리: 실제 성공률(60%)·속도(40%)로 각 모델을 지속 채점하고 동적으로 최적 후보를 선택합니다.

이점: 실제 런타임 텔레메트리를 기반으로 응답 품질과 속도를 자동 최대화합니다.

비용과 그림자(트레이드오프): 탐색률(Epsilon, 예: 0.15 = 15%)로 라우터가 고의로 계속 다른 모델을 시도하므로 개별적으로 느릴 수 있습니다.

언제 비활성화합니까? 및 임곗값: 매 실행 100% 고정 예측 가능 모델을 강제해야 하면 끄십시오(또는 Epsilon 을 0.00 으로). | 임계값: Epsilon: 0.00 = 최선 모델 고수 | > 0.30 = 약한 모델 빈번 시도 (기본값: 0.15). 성공/지연 가중치: 기본값 0.60 / 0.40.

112. 동적 임시 팀 스포닝 (backend/ephemeral_team_spawner.py)

문제: 단일 제너럴리스트 에이전트는 대규모 횡단 과제(아키텍처+UI+보안+QA)에 금방 과부하됩니다.

작동 원리: 미션 브리핑에 맞춰 임시 전문가 태스크포스를 모집하고 임무 완료 후 다시 해산합니다.

이점: 지속적인 리소스 오버헤드 없이 필요에 따라 전문 지식을 묶습니다.

비용과 그림자(트레이드오프): 4 인 임시 팀은 토큰 소비를 배가시키고(3~5 배) 팀원 간 추가 조정 오버헤드가 필요합니다.

언제 비활성화합니까? 및 임곗값: 단순 개별 작업·부족한 토큰 예산 시 끄십시오. 일상 최대 팀 크기는 3–4 로 제한하십시오. | 임계값: 최대 팀 크기: 2–4 = 이상적 | > 5 = 높은 통신 오버헤드·5 배 토큰 소비 (기본값: 4).

113. 라이브 리소스 보호기 및 호스트 백프레셔(backend/resource_backpressure.py)

문제: 병렬 컨테이너, 브라우저 감사, 로컬 프로세스는 피크 부하 시 호스트 RAM·CPU 를 과부하시킬 수 있습니다(OOM-Kill).

작동 원리: CPU·RAM·디스크 지표를 실시간 모니터링하고 85% 부하부터 저우선순위 백그라운드 작업을 자동 스로틀합니다.

이점: 과부하 충돌로부터 서버를 안정적으로 보호하고 핵심 프로세스를 항상 반응 상태로 유지합니다.

비용과 그림자(트레이드오프): 임계값이 너무 낮으면(CPU < 70% 또는 RAM < 70%) 짧은 정상 피크에도 백그라운드 작업이 계속 스로틀됩니다.

언제 비활성화합니까? 및 임곗값: 완전히 끄지 말고 강력한 하드웨어에서 CPU/RAM 임계값을 90%로 올리십시오. | 임계값: Max CPU/RAM: < 70% = 정상 작업 스로틀 | > 92% = OOM 위험 (기본값: 85%). 최소 여유 디스크: 기본값 2.0GB.

18. 축 3: 경제 및 ERP(알파) 및 의뢰인 스위트

모듈 / 화면	기능 범위 및 기술적 특징(v0.5.1)
주소 및 CRM + PocketBase 동기화	중앙 연락처·채무자(10000+)·채권자(70000+) 원장, 360° 연락처 서랍, 멀티채널 타임라인, 캘린더/재제출, PocketBase 경유 LinkedIn·미디어 동기화(Port 3005/8090/8092).
품목 및 용역(ART)	ART-1001 부터 서비스, 시간당 요금, 패키지, SaaS 구독, 재고 관리 실물 제품의 마스터 카탈로그.
업무 및 계약	연속 증빙 체인 제안(ANG-YYYY-NNNN) → 주문(AB-YYYY-NNNN), 가중 딜 파이프라인, 매출 예측, 계약 기한 레이더(30/14/7 일 알람) 포함.
회계 및 DATEV	§ 14 UStG 매출·매입 증빙, GiroCode 결제 QR, 재무 분석, 자동 독촉(Dunning Engine), DATEV 용 EXTF/CSV 내보내기.
기업 및 의뢰인 격리	여러 자사 사업(company_id)을 WAL 모드의 물리 분리 SQLite DB, 자체 번호대, 서신지, 템플릿 편집기로 운영합니다.

19. 아키텍처, 보안 및 배포 기능(v0.5.0 – v0.5.1)

기능 / 하위 시스템	구현 및 운영 효용(v0.5.1)
모듈형 라우터 아키텍처	archon/routers/ 전용 FastAPI 라우터로 분리(main.py 를 16,254 줄에서 1,875 줄로 축소, -88%), 중앙 설정 계층 archon/core/config.py.
보안 강화(Archon Hardened)	25 개 중요·68 개 높음 감사 finding 종결, Starlette 스레드풀 경감, 강화 UI DOM 경로, hermes_updater 용 클래스 A Docker 관리 프록시.
RBAC 사용자 관리(17.10)	backend/auth_service.py 경유 세분 역할(admin, operator, viewer), 100,000 회 PBKDF2-HMAC-SHA256 비밀번호 해싱, 단일/다중 사용자 모드.
2 단계 시스템 업데이트(17.11)	?local_only=true 로 로컬 버전/HEAD 즉시 표시(약 0.25 초), 비동기 GitHub 동기화, 업데이트 전 SQLite 온라인 스냅샷, 원클릭 롤백.
AnythingLLM Proxy 및 Chroma 스택	/anythingllm-proxy 경유 AnythingLLM 을 중앙 라이브러리에 매끄럽게 내장(/assets/* Fast-Path, 로컬 ChromaDB 벡터 인덱스 포함).
휴대용 1-클릭 설치 프로그램	setup.bat, setup.ps1, install.sh 경유 플랫폼 횡단 최초 설치(docker-compose.yml, .env.example, INSTALL_AGENT.md, 9 개 언어 GitHub 문서).
4-아이 품질 게이트 및 코드 샌드박스	티켓 종료 전 Kanban 자동 2 차 검사와 생성 아티팩트 실시간 미리보기용 Showroom 인터랙티브 코드 샌드박스(4.4.1).

20. 가나다순 용어집 및 색인

핵심 모듈·아키텍처 구성 요소·기능(KO)의 알파벳 용어집(정확한 페이지 번호 포함).

기능 / 용어	간략한 설명	페이지
3 계층 비용 라우팅	작업 복잡도 및 예산에 따른 LOW / MEDIUM / HIGH 자동 모델 선택.	S. 3, 8
4-아이 품질 게이트	감사자 에이전트에 의한 완료된 칸반 작업의 독립적 검토.	S. 3, 14, 31
Anti-AI-Slop 및 STE100	불필요한 상투어 방지 및 출력 토큰 절감을 위한 스타일/단어 필터.	S. 8, 10
AnythingLLM Proxy	중앙 라이브러리에서 /anythingllm-proxy 를 통한 AnythingLLM + ChromaDB 로컬 임베딩.	S. 3, 31
Archon Hardened (v0.5.1)	모듈형 라우터 구조와 종결된 감사 소견의 보안/안정성 릴리스.	S. 3, 31
Bandit 모델 라우터	적중률 및 비용 기반 동적 모델 선택용 적응형 라우터(Multi-Armed Bandit).	S. 29, 30
ChromaDB	제 2 기둥에서 시맨틱 문서 검색을 위한 로컬 벡터 데이터베이스.	S. 25, 31
Corrective RAG (CRAG)	답변 생성 전 관련성 평가를 포함한 자기 검증 지식 조회.	S. 25, 26
DAG 스케줄러	병렬 Fan-Out / Fan-In 을 적용한 종속 하위 작업의 2 단계 실행 계획.	S. 12, 13, 24
DATEV 내보내기 및 GiroCode	회계 데이터용 EXTF/CSV 인터페이스 및 송장용 EPC 결제 QR.	S. 3, 31
Dead-Letter-Queue (DLQ)	실패한 작업의 자동 재시도 제어가 있는 오류 대기열.	S. 27, 28
Decision Engine(55 개 프레임워크)	9 개 방법론 카테고리에 걸친 구조화된 보드룸 토론용 전략 모듈.	S. 3, 5, 20
DSPy / TextGrad Optimizer	실제 평가 지표 기반 시스템 프롬프트 자동 개선.	S. 29
ERP 증빙 체인(ANG -> AB -> RE)	파이프라인 전망 및 독촉 관리가 포함된 연결된 영업 및 회계 문서.	S. 22, 24, 25, 31
Evey / Hermes 플러그인	메모리, 위임, 검증 및 인터페이스용 모듈형 시스템 확장.	S. 3, 19, 20, 21
Hermes Master Orchestrator	포트 8420 데몬에서 목표 분해, 위임 및 통합용 중앙 코어 엔진.	S. 9, 11, 12, 19
JEV 라우터(0-토큰 인텐트)	명확한 단축 명령을 위한 AI 비용 없는 결정적 사전 필터.	S. 8, 10
MCP (Model Context Protocol)	안전한 도구 연결을 위한 표준화된 드라이버 아키텍처.	S. 6, 14
OmniRoute AI Gateway	무료 등급 라우팅, 지연 헤징 및 평가 아레나를 포함한 멀티 프로바이더 게이트웨이.	S. 3, 6, 21
PBKDF2 / Argon2id RBAC	System Control Center(탭 17.10)에서 역할 및 비밀번호 보호.	S. 3, 28, 29, 31
PII 스크러버 및 재수화	외부 LLM 호출 전 민감한 개인정보의 양방향 비식별화.	S. 3, 28

기능 / 용어	간략한 설명	페이지
Research Lab	근거 삼각측량을 적용한 다단계 웹 및 출처 리서치.	S. 3, 7, 21
SLA 계약(Contract Enforcement)	전문화된 에이전트 간의 정식 인계 및 품질 보증.	S. 13, 23, 24
SQLite WAL 연결 풀	Write-Ahead Logging 을 통한 쓰기 안전한 멀티스레드 데이터베이스 액세스.	S. 15, 20, 21, 24
System Control Center(11 개 탭)	사용자/RBAC 및 2 단계 시스템 업데이트를 포함한 중앙 관리 콘솔.	S. 3, 8, 11, 31
Token Guard 및 Loop-Killer	무한 루프(Jaccard 검사) 및 토큰 초과에 대한 실시간 보호.	S. 3, 6, 8, 11
Watchdog 및 데드락 슈퍼바이저	정지된 작업, 컨테이너의 30 초 모니터링 및 자동 자가 치유.	S. 6, 11, 12, 21
감사 로그(SHA-256 해시 체인)	모든 시스템 및 에이전트 조치의 감사 대응 기록 체인.	S. 13, 17, 28, 29
기업 및 의뢰인 지원 기능	company_id 별 여러 법인의 물리적 데이터베이스 분리.	S. 3, 31
듀러블 실행 및 타임 트래블	SQLite 체크포인트 및 단계별 실행 리플레이가 있는 중단 가능한 장기 실행.	S. 3, 21, 27, 28
라우터 아키텍처(archon/routers)	슬림한 FastAPI 라우터로의 모듈형 백엔드 분할(main.py 파일 크기 - 88%).	S. 3, 31
멱등성 키 및 드라이런	중복 도구 실행 방지 및 쓰기 액세스 없는 시뮬레이션 모드.	S. 27, 28
봇 스크린(웹 내비게이터)	Playwright 기반 브라우저 상호작용용 시각적 실행 모니터.	S. 14, 23
슬립 사이클 통합	야간 중복 제거와 에이전트 기억의 장기 압축.	S. 15, 23
승인 게이트(HITL)	외부 이메일, 예약 또는 중요 파일 액세스 전 인간 검토 단계.	S. 4, 17, 22, 28
시맨틱 캐싱	토큰 절약을 위한 유사 프롬프트 24 시간 중간 저장.	S. 8, 9, 24, 26
이그레스 방화벽	에이전트 도구 호출용 아웃바운드 네트워크 및 도메인 제어.	S. 28
이벤트 메시 및 웹훅 트리거	수신 웹훅과 폴더 변경 사항의 이벤트 기반 처리.	S. 14, 21, 22
임시 코드 샌드박스	안전한 Python·스크립트 코드용 휘발성 격리 컨테이너 런타임.	S. 3, 23, 31
지식 그래프	벡터 데이터 외 구조화된 엔터티 관계용 로컬 트리플 저장소.	S. 25, 26
컨텍스트 압축 및 LRU-K	RAM 한도 도달 시 긴 채팅 컨텍스트의 스왑 아웃 및 압축.	S. 8, 9, 24, 25
프리모템 및 합의 게이트	중요 승인 전 시뮬레이션 오류 스트레스 테스트 및 다중 에이전트 조율.	S. 5, 21, 23, 24
프리플라이트 비용 추정기	초과 시 작업 예산 중단이 포함된 예상 토큰 비용 사전 계산.	S. 8, 10, 11

기능 / 용어	간략한 설명	페이지
하이브리드 검색(FTS5 + RRF)	BM25 전문 검색과 Reciprocal Rank Fusion 기반 벡터 검색의 조합.	S. 25